

Местоположение, държави и настройки الأذان

Приложението определя само къде си. В България показва селото или града и часовете по Мюфтийството, а извън България изчислява часовете за точното място, включително на северни ширини. Добавени са екранът „Настройки“ и изборът на населено място с търсене.

expo-location

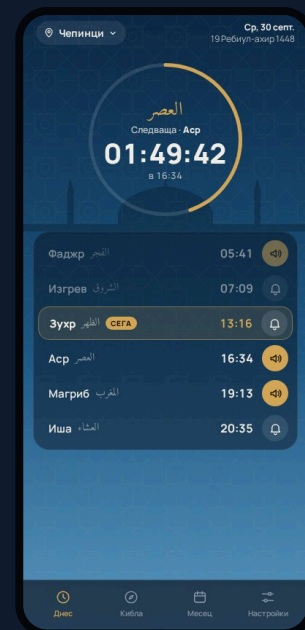
6 730 населени места

Граница на България

Настройки

Jest – 100 теста

37 файла



Какво е решено в този етап

GPS само докато приложението е отворено · „За приложението“ с източниците и поверителността · в България: населеното място, в което си (OpenStreetMap; в целия град – градът) + календарът на Мюфтийството (Чепинци = София –6 мин.) · извън България: изчисление, метод по избор, Шафии/Ханафи, правило за северни ширини над 48° · ръчни корекции ±15 мин. и дата по Хиджра ±2 дни · обновяването на календара от GitHub е отложено („Поддръжка“).

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нов пакет и нов build

03 Данни: населени места и граница

04 Логика: къде съм и откъде са часовете

05 Настройки и GPS

06 Екраните

07 Скорост и индикатори за зареждане

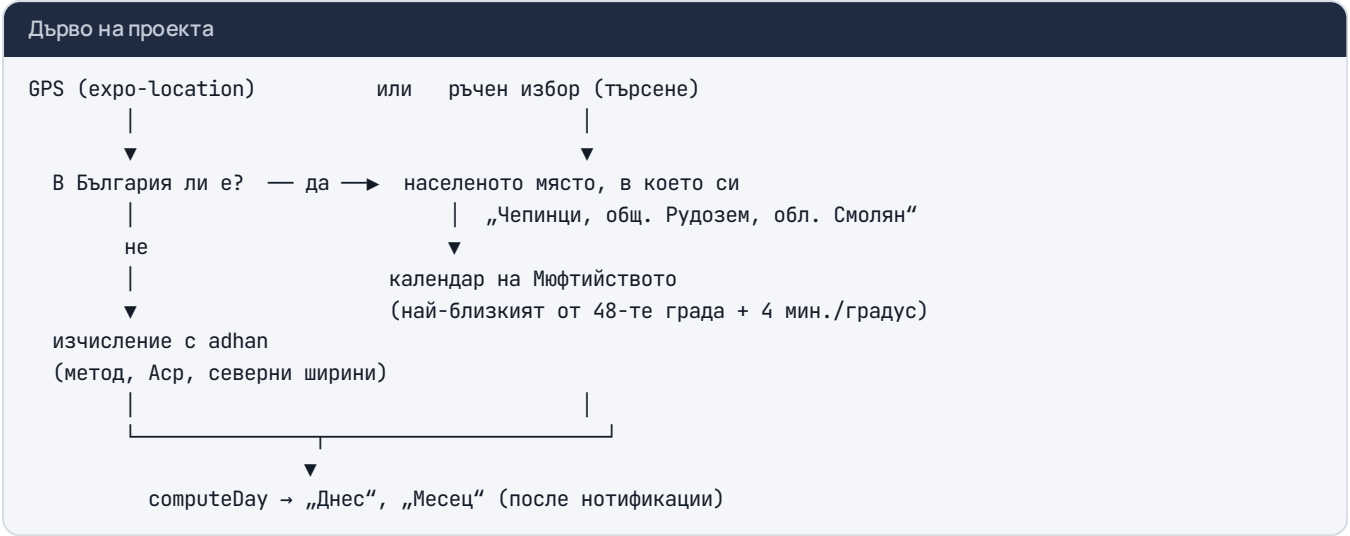
08 Тестове

09 Проверка и край на етапа

Стъпка 1

Какво прави етапът

Един и същ път за всички екрани: **координати** → **място** → **източник на часовете** → **часове**.



ПРИМЕР	ИЗТОЧНИК	РЕЗУЛТАТ (30.09.2026)
Чепинци (Смолянско)	Мюфтийство, София -6	05:41 · 07:09 · 13:16 · 16:34 · 19:13 · 20:35
Истанбул	изчисление, Диянет	часове за точните координати
Единбург	изчисление + „1/7 от нощта“	Фаджр и Иша не се сливат през лятото

Стъпка 2

Нов пакет и нов build

`expo-location` има native код, затова след инсталирането трябва нов development build.

Инсталиране и нов build

```
npx expo install expo-location
Remove-Item -Recurse -Force android
npx expo run:android
```

`app.json` корен на проекта

- Plugin-ът на `expo-location` иска само достъп „докато приложението се ползва“. Фоновият достъп и сензорът за движение са изрично изключени (`false`). Това улеснява одобрението в магазините.
- Текстът на въпроса за разрешение на iOS е на английски тук, а на български – в `locales/bg.json` .

app.json83 реда

```
{
  "expo": {
    "name": "Adhan",
    "slug": "adhan",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "../assets/icon.png",
```

```

"userInterfaceStyle": "dark",
"ios": {
  "supportsTablet": false,
  "bundleIdentifier": "com.ilkoadamov.adhan",
  "infoPlist": {
    "CFBundleAllowMixedLocalizations": true
  }
},
"android": {
  "adaptiveIcon": {
    "backgroundColor": "#0F1B2D",
    "foregroundImage": "./assets/android-icon-foreground.png",
    "backgroundImage": "./assets/android-icon-background.png",
    "monochromeImage": "./assets/android-icon-monochrome.png"
  },
  "predictiveBackGestureEnabled": false,
  "package": "com.ilkoadamov.adhan"
},
"web": {
  "favicon": "./assets/favicon.png"
},
"plugins": [
  "expo-router",
  [
    "expo-splash-screen",
    {
      "backgroundColor": "#0F1B2D",
      "image": "./assets/splash-icon.png",
      "imageWidth": 160
    }
  ],
  "expo-font",
  [
    "expo-build-properties",
    {
      "android": {
        "cmakeVersion": "3.31.6"
      }
    }
  ],
  [
    "expo-localization",
    {
      "supportedLocales": {
        "ios": [
          "en",
          "bg"
        ],
        "android": [
          "en",
          "bg"
        ]
      }
    }
  ],
  [
    "expo-location",
    {
      "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place where you are.",
      "locationAlwaysAndWhenInUsePermission": false,

```

```

        "locationAlwaysPermission": false,
        "motionUsagePermission": false
    }
  ],
  "scheme": "adhan",
  "backgroundColor": "#0F1B2D",
  "experiments": {
    "typedRoutes": true
  },
  "locales": {
    "bg": "./locales/bg.json",
    "en": "./locales/en.json"
  }
}

```

bg.json locales

- `NSLocationWhenInUseUsageDescription` – обяснението, което iPhone показва при искане на разрешение. Android не показва такъв текст.

locales/bg.json

9 реда

```

{
  "ios": {
    "CFBundleDisplayName": "Езан",
    "NSLocationWhenInUseUsageDescription": "Езан използва местоположението ти, за да покаже часовете за намаз за мястото, където се намиращ."
  },
  "android": {
    "app_name": "Езан"
  }
}

```

Стъпка 3

Данни: населени места и граница

Два нови файла в `src/data`. И двата работят без интернет.

src/data/places.json (320 KB; областите и общините са индекси, за да е по-малък)

```

{
  "source": "OpenStreetMap – © OpenStreetMap contributors, лиценз ODbL",
  "oblasti": ["Благоевград", "Бургас", ...], // 28 области
  "obshtini": ["Айтос", "Аксаково", ...], // 264 общини
  "places": [
    ["Чепинци", 41.4348, 24.8682, 20, 189, 1], // име, ширина, дължина, област, община, вид
    ... // 6 730 места
  ]
}

```

src/data/bulgaria-border.json (10 KB)

```
{
  "source": "Natural Earth 1:10m (public domain), опростено до ~200 м",
  "ring": [[22.3573, 42.3137], [22.3629, 42.3212], ...] // 591 точки: [дължина, ширина]
}
```

- **Вид:** 0 – град (256), 1 – село (4 857), 2 – махала (1 617).
- **Без местности, вилни зони и квартали.** В OpenStreetMap някои местности са отбелязани като махали („м. Детски град“, „в.з. Боровете“, „кв. ...“). Те не са населени места и са махнати от файла (181 записа).
- **Лиценз ODbL:** позволява ползване в приложението, ако е посочено „© OpenStreetMap contributors“ – вижда се в „Настройки → Източници“.
- Границата е опростена до ~200 м. Крайбрежието при опростяването „изрязва“ някои места (Балчик, Равда) – затова има и втора проверка (стъпка 4).

Стъпка 4

Логика: къде съм и откъде са часовете

Всичко е чист TypeScript в `src/domain` и е покрито с тестове.

translit.ts `src/domain`

- Официалната транслитерация: „Чепинци“ → „Chepintsi“. Ползва се за английските имена и за търсене с латиница.
- „София“ е изключение – „Sofia“, а не „Sofiya“.

src/domain/translit.ts

32 реда

```
/**
 * Официалната транслитерация на български (Закон за транслитерацията, 2009):
 * „Чепинци“ → „Chepintsi“, „Кърджали“ → „Kardzhali“, „-ия“ в края на думата → „-ia“.
 * Ползва се за имената на градовете на английски и за търсене с латиница.
 */
const MAP: Record<string, string> = {
  а: 'a', б: 'b', в: 'v', г: 'g', д: 'd', е: 'e', ж: 'zh', з: 'z', и: 'i', й: 'y',
  к: 'k', л: 'l', м: 'm', н: 'n', о: 'o', п: 'p', р: 'r', с: 's', т: 't', у: 'u',
  ф: 'f', х: 'h', ц: 'ts', ч: 'ch', ш: 'sh', щ: 'sht', ъ: 'a', ъ: 'y', ю: 'yu', я: 'ya',
};

/** Имена с утвърдено английско изписване, различно от правилото. */
const EXCEPTIONS: Record<string, string> = { София: 'Sofia' };

function word(w: string): string {
  const lower = w.toLowerCase();
  let out = '';
  for (const ch of lower) out += MAP[ch] ?? ch;
  if (lower.endsWith('ия')) out = out.slice(0, -3) + 'ia';
  // главна буква, ако оригиналът започва с главна
  return w[0] && w[0] !== w[0].toLowerCase() ? out.charAt(0).toUpperCase() + out.slice(1) : out;
}

export function transliterate(text: string): string {
  if (EXCEPTIONS[text]) return EXCEPTIONS[text];
  return text.split(/(\s|-)/).map((part) => (/^[^s-]+$/.test(part) ? part : word(part))).join('');
```

```

}

/** За търсене: малки букви, без точки и двойни интервали, „й“ → „И“. */
export function normalizeForSearch(text: string): string {
  return text.toLowerCase().replace(/й/g, 'И').replace(/[.,]/g, '').replace(/\s+/g, ' ').trim();
}

```

geo.ts src/domain

- **insideBorder** – класическият алгоритъм „лъч“: брой колко пъти хоризонтална линия от точката пресича границата. Нечетен брой = вътре.
- **distanceKm** е преместена тук от **mufti.ts**, за да се ползва и от населените места.

src/domain/geo.ts

44 реда

```

import border from '@data/bulgaria-border.json';

/**
 * Проверка „в България ли е тази точка“ – без интернет.
 * Границата е от Natural Earth 1:10m (public domain), опростена до ~200 м (≈10 KB).
 */
const RING: readonly (readonly number[])[] = border.ring; // [дължина, ширина]

export function distanceKm(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const rad = Math.PI / 180;
  const dLat = (lat2 - lat1) * rad;
  const dLon = (lon2 - lon1) * rad;
  const a =
    Math.sin(dLat / 2) ** 2 + Math.cos(lat1 * rad) * Math.cos(lat2 * rad) * Math.sin(dLon / 2) ** 2;
  return 6371 * 2 * Math.asin(Math.sqrt(a));
}

/**
 * Бързо разстояние за търсене на най-близкото място: равногълна проекция,
 * без тригонометрия в цикъла (cosLat се смята веднъж за точката).
 * На разстояния до стотина км грешката е под 0,5 % – за избор на населено място е без значение.
 */
export function fastKm(lat1: number, lon1: number, lat2: number, lon2: number, cosLat: number): number {
  const dy = lat2 - lat1;
  const dx = (lon2 - lon1) * cosLat;
  return 111.195 * Math.sqrt(dx * dx + dy * dy);
}

/** Правогълникът около България (с ~5 км запас): извън него точката със сигурност не е в България. */
export function nearBulgaria(lat: number, lon: number): boolean {
  return lat > 41.18 && lat < 44.27 && lon > 22.3 && lon < 28.7;
}

/** Класическият алгоритъм „лъч“: броим колко пъти хоризонтален лъч пресича границата. */
export function insideBorder(lat: number, lon: number): boolean {
  let inside = false;
  for (let i = 0, j = RING.length - 1; i < RING.length; j = i++) {
    const [xi, yi] = RING[i];
    const [xj, yj] = RING[j];
    const crosses = yi > lat !== yj > lat && lon < ((xj - xi) * (lat - yi)) / (yj - yi) + xi;
    if (crosses) inside = !inside;
  }
  return inside;
}

```

places.ts src/domain

- `isInBulgaria` : вътре в границата **или** на под 1,5 км от българско населено място. 1,5 км хваща плажовите и крайбрежието, но не и Промахон (Гърция), който е на 2,1 км от Кулата.
- `placeAt` – в кое населено място си по GPS. Всяко място в OpenStreetMap е само точка (центърът), затова всяко има приблизителен радиус: София 12 км, Пловдив 6, Варна 7 ... други градове 2, село 1, махала 0,4 км. Ако си в радиуса на няколко места, печели онова, спрямо което си „най-навътре“ (разстояние / радиус): в Младост → София, в Кубратово (в радиуса на София, но до центъра на селото) → Кубратово. Ако не си в ничий радиус – мястото с най-близък край.
- `placeToLocation` прави от населено място `AppLocation` с име, уточнение („общ. Рудозем, обл. Смолян“) и разликата спрямо София по правилото на Мюфтийството.
- `searchPlaces` : точно съвпадение → започва с → съдържа; при равенство градовете са преди селата, после по азбука. Търси и на кирилица, и на латиница.
- **Без `localeCompare` по време на работа.** В Hermes (JS машината на телефона) всяко `localeCompare(..., 'bg')` създава нов collator и е много бавно: сортирането на 875-те резултата за „ch“ замразяваше екрана за секунди. Затова `places.json` е подреден по азбука още при подготовката на данните, а тук при равенство се сравнява само индексът. `allCities` и `warmSearchIndex` също са тук.

src/domain/places.ts

256 реда

```
import data from '@data/places.json';

import { fastKm, insideBorder, nearBulgaria } from './geo';
import type { AppLocation } from './location';
import { muftiShiftFor } from './mufti';
import { normalizeForSearch, transliterate } from './translit';

/**
 * Всички ~6 700 населени места в България (OpenStreetMap, © OpenStreetMap contributors, ODbL).
 * Във файла: [име, ширина, дължина, индекс на област, индекс на община, вид].
 * Местностите („м. ...“), вилните зони и кварталите са махнати при подготовката
 * на файла – те не са населени места (оттам идваше „м. Детски град“ в София).
 */
export type PlaceKind = 0 | 1 | 2; // град, село, махала

export interface Place {
  index: number;
  name: string;
  lat: number;
  lon: number;
  oblast: string;
  obshtina: string;
  kind: PlaceKind;
}

type Row = [string, number, number, number, number, number];
const ROWS = data.places as Row[];

export const PLACES_SOURCE: string = data.source;
export const PLACES_COUNT = ROWS.length;

export function getPlace(index: number): Place {
  const [name, lat, lon, o, s, kind] = ROWS[index];
  return {
    index,
```

```

    name,
    lat,
    lon,
    oblast: data.oblasti[o],
    obshtina: data.obshtini[s],
    kind: kind as PlaceKind,
  };
}

/** Най-близката точка от списъка по права линия (за проверката „в България ли съм“). */
export function nearestPlace(lat: number, lon: number): { place: Place; km: number } {
  let best = 0;
  let bestKm = Infinity;
  const cosLat = Math.cos((lat * Math.PI) / 180);
  for (let i = 0; i < ROWS.length; i++) {
    const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
    if (km < bestKm) {
      best = i;
      bestKm = km;
    }
  }
  return { place: getPlace(best), km: bestKm };
}

/**
 * Приблизителен радиус (км) на населеното място около точката му в OpenStreetMap.
 * Всяко място е само една точка (центърът), а големият град е широк километри:
 * в Младост си на 9 км от центъра на София и по-близо до точката на село Герман.
 */
const CITY_RADIUS_KM: Record<string, number> = {
  София: 12,
  Пловдив: 6,
  Варна: 7,
  Бургас: 6,
  Русе: 5,
  'Стара Загора': 4,
  Плевен: 4,
  Сливен: 3.5,
  Добрич: 3.5,
  Шумен: 3.5,
  Перник: 3.5,
  Хасково: 3.5,
  Ямбол: 3.5,
  Пазарджик: 3.5,
  Благоевград: 3.5,
  'Велико Търново': 3.5,
  Габрово: 4,
  Смолян: 4,
  Кърджали: 3,
  Кюстендил: 3,
  Враца: 3,
  Видин: 3,
  Монтана: 3,
  Ловеч: 3,
  Силистра: 3,
  Търговище: 3,
  Разград: 3,
  Казанлък: 3,
  Асеновград: 3,
  Дупница: 3,
  'Горна Оряховица': 3,

```

```

    Димитровград: 3,
};
const TOWN_RADIUS_KM = 2;
const VILLAGE_RADIUS_KM = 1;
const HAMLET_RADIUS_KM = 0.4;

function radiusKm(row: Row): number {
    if (row[5] === 0) return CITY_RADIUS_KM[row[0]] ?? TOWN_RADIUS_KM;
    return row[5] === 1 ? VILLAGE_RADIUS_KM : HAMLET_RADIUS_KM;
}

/**
 * В кое населено място съм по GPS.
 * 1. Ако съм в радиуса на едно или повече места – мястото, спрямо което съм
 * „най-навътре“ (разстояние / радиус). В Младост → София; в Кубратово,
 * което е в радиуса на София, но на 0,2 км от центъра си → Кубратово.
 * 2. Иначе – мястото с най-малко разстояние до края му (разстояние - радиус).
 */
export function placeAt(lat: number, lon: number): Place {
    let inside = -1;
    let insideRatio = Infinity;
    let outside = 0;
    let outsideGap = Infinity;
    const cosLat = Math.cos((lat * Math.PI) / 180);
    for (let i = 0; i < ROWS.length; i++) {
        const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
        const r = radiusKm(ROWS[i]);
        if (km <= r) {
            if (km / r < insideRatio) {
                inside = i;
                insideRatio = km / r;
            }
        } else if (km - r < outsideGap) {
            outside = i;
            outsideGap = km - r;
        }
    }
    return getPlace(inside >= 0 ? inside : outside);
}

/**
 * В България ли е точката. Границата сама не стига: опростената брегова линия
 * „изразява“ някои плажове и крайбрежни селища (Балчик, Равда). Затова точка,
 * която е на под 1,5 км от българско населено място, също се брои за България.
 * 1,5 км е под разстоянието от Промахон (Гърция) до Кулата (2,1 км).
 */
export function isInBulgaria(lat: number, lon: number): boolean {
    if (!nearBulgaria(lat, lon)) return false; // Истанбул, Единбург... – без да обхождаме 6 700 места
    if (insideBorder(lat, lon)) return true;
    return nearestPlace(lat, lon).km < 1.5;
}

/** Името на мястото на двата езика и уточнение (община, област) – за различаване на еднакви имена. */
export function placeToLocation(place: Place): AppLocation {
    const cityLike = place.kind === 0;
    return {
        id: `bg-${place.index}`,
        names: { bg: place.name, en: transliterate(place.name) },
        detail: {
            bg: cityLike ? `обл. ${place.oblast}` : `общ. ${place.obshtina}, обл. ${place.oblast}`,
            en: cityLike

```

```

        ? `${transliterate(place.oblast)} Province`
        : `${transliterate(place.obshtina)}, ${transliterate(place.oblast)} Province`,
    },
    latitude: place.lat,
    longitude: place.lon,
    source: 'mufti',
    muftiShift: muftiShiftFor(place.lat, place.lon),
  };
}

/** София-град от списъка - мястото по подразбиране, докато няма GPS или избор. */
export function defaultPlace(): Place {
  const i = ROWS.findIndex((r) => r[0] === 'София' && r[5] === 0);
  return getPlace(i);
}

// Индексът за търсене: 6 700 транслитерации - на компютъра ~50 ms, на телефона
// няколкостотин ms. Строи се на части, за да не блокира екрана (виж warmSearchIndex).
const searchIndex: { bg: string; lat: string }[] = [];
const CHUNK = 500;

function buildMore(count: number): void {
  const end = Math.min(ROWS.length, searchIndex.length + count);
  for (let i = searchIndex.length; i < end; i++) {
    searchIndex.push({
      bg: normalizeForSearch(ROWS[i][0]),
      lat: normalizeForSearch(transliterate(ROWS[i][0])),
    });
  }
}

export function isSearchIndexReady(): boolean {
  return searchIndex.length === ROWS.length;
}

function index() {
  if (!isSearchIndexReady()) buildMore(ROWS.length); // търсене преди края - довършва наведнъж
  return searchIndex;
}

let warming: Promise<void> | null = null;
/**
 * Строи индекса на части от по 500 места, с пауза между тях - екранът и клавиатурата
 * остават живи. Вика се при отваряне на екрана за търсене; обещанието се изпълнява, когато е готов.
 */
export function warmSearchIndex(): Promise<void> {
  if (isSearchIndexReady()) return Promise.resolve();
  if (!warming) {
    warming = new Promise((resolve) => {
      const step = () => {
        buildMore(CHUNK);
        if (isSearchIndexReady()) resolve();
        else setTimeout(step, 0);
      };
      setTimeout(step, 0);
    });
  }
  return warming;
}

/**

```

```

* Подредбата по азбука е направена в places.json при подготовката на данните.
* Тук НЕ се ползва localeCompare: в Hermes (JS машината на телефона) всяко
* localeCompare(..., 'bg') създава нов collator и е много бавно – сортирането на
* 875 резултата за „ch“ замразяваше екрана за няколко секунди.
* Затова при равенство се сравнява само индексът (= азбучният ред).
*/

let cities: Place[] | null = null;
/** Всички 256 града по азбучен ред – показват се, докато няма търсене. */
export function allCities(): Place[] {
  if (!cities) {
    cities = [];
    for (let i = 0; i < ROWS.length; i++) if (ROWS[i][5] === 0) cities.push(getPlace(i));
  }
  return cities;
}

/**
* Търсене по име на кирилица или латиница („чеп“, „cher“).
* Подредба: точно съвпадение → започва с → съдържа; при равенство – градове
* преди села, после по азбука.
*/
export function searchPlaces(query: string, limit = 40): Place[] {
  const q = normalizeForSearch(query);
  if (q.length < 2) return [];
  const idx = index();
  const hits: { i: number; rank: number }[] = [];
  for (let i = 0; i < idx.length; i++) {
    const { bg, lat } = idx[i];
    let rank = -1;
    if (bg === q || lat === q) rank = 0;
    else if (bg.startsWith(q) || lat.startsWith(q)) rank = 1;
    else if (bg.includes(q) || lat.includes(q)) rank = 2;
    if (rank >= 0) hits.push({ i, rank: rank * 10 + ROWS[i][5] });
  }
  hits.sort((a, b) => a.rank - b.rank || a.i - b.i);
  return hits.slice(0, limit).map((h) => getPlace(h.i));
}

```

location.ts src/domain

- Ново поле `detail` – уточнението под името. Извън България там е държавата.
- `formatCoords` – „55.95°N 31.9°W“, когато извън България няма интернет за името на мястото.

src/domain/location.ts

26 реда

```

/**
* Откъде идват часовете за едно място:
* - 'mufti' – официалният календар на Главно мюфтийство (всички места в България);
* - 'calc' – астрономическо изчисление с adhan (извън България).
*/
export type TimeSource = 'mufti' | 'calc';

export interface AppLocation {
  id: string;
  /** Името на мястото на двата езика (градовете са данни, а не текстове на интерфейса). */
  names: { bg: string; en: string };
  /** Уточнение под името: „общ. Рудозем, обл. Смолян“ или държавата извън България. */
  detail?: { bg: string; en: string };
  latitude: number;
}

```

```

longitude: number;
source: TimeSource;
/** Само за 'mufti': разлика в минути спрямо таблицата на София (Варна = -18). */
muftiShift?: number;
}

/** „55.95°N 3.19°W“ – когато извън България няма име на мястото (без интернет). */
export function formatCoords(lat: number, lon: number): string {
  const ns = lat >= 0 ? 'N' : 'S';
  const ew = lon >= 0 ? 'E' : 'W';
  return `${Math.abs(lat).toFixed(2)}°${ns} ${Math.abs(lon).toFixed(2)}°${ew}`;
}

```

abroadName.ts src/domain

- Името извън България от обратното геокодиране на телефона: на първия ред **винаги градът**, на втория – районът и държавата: „İstanbul“ / „Fatih, Türkiye“, „Edinburgh“ / „Leith, United Kingdom“.
- Турция е отделен случай: там градът е областта (**region**), а Android често връща района като **city** („Fatih“). Централният район „Merkez“ не се показва.
- Другаде градът е **city** (ако липсва – окръгът, после областта), а районът – **district**. Повторения като „City of Edinburgh“ се пропускат.

src/domain/abroadName.ts

76 реда

```

import type { PlaceName } from './resolve';

/**
 * Полетата от обратното геокодиране на телефона, които ни трябва
 * (подмножество на LocationGeocodedAddress от expo-location).
 * Android (Google): city = locality, district = sublocality, subregion = subAdminArea, region = adminArea.
 * iOS (Apple):      city = locality, district = sublocality, subregion = subAdministrativeArea, region =
 * administrativeArea.
 */
export interface GeocodedAddress {
  city?: string | null;
  district?: string | null;
  subregion?: string | null;
  region?: string | null;
  country?: string | null;
  isoCountryCode?: string | null;
}

/** За сравнение: без главни букви и диакритика („İstanbul“ = „Istanbul“). */
function norm(s: string): string {
  return s
    .normalize('NFD')
    .replace(/[\i]/g, '')
    .replace(/ı/g, 'i')
    .toLowerCase()
    .trim();
}

function clean(s: string | null | undefined): string | null {
  const t = s?.trim();
  return t ? t : null;
}

/** Първата стойност, която я има и не е (част от) името на града. */

```

```

function firstOther(main: string, values: (string | null | undefined)[]): string | null {
  const m = norm(main);
  for (const v of values) {
    const c = clean(v);
    if (!c) continue;
    const n = norm(c);
    if (n === m || n.includes(m) || m.includes(n)) continue; // „City of Edinburgh“ до „Edinburgh“
    return c;
  }
  return null;
}

/**
 * Име на място извън България: на първия ред винаги градът,
 * на втория – районът (ако го има) и държавата: „İstanbul“ / „Fatih, Türkiye“.
 *
 * Турция е отделен случай: там областта (il) е градът, а „city“ от
 * геокодирането често е районът (ilçe) – в Истанбул Android връща „Fatih“.
 * Затова за Турция градът е region, а районът – subregion или city.
 * Другаде градът е city (ако го няма – subregion, после region),
 * а районът – district (квартал / част от града).
 */
export function nameFromAddress(addr: GeocodedAddress | null | undefined): PlaceName | null {
  if (!addr) return null;
  const turkey = addr.isoCountryCode?.toUpperCase() === 'TR';

  const city = turkey
    ? clean(addr.region) ?? clean(addr.city) ?? clean(addr.subregion)
    : clean(addr.city) ?? clean(addr.subregion) ?? clean(addr.region);
  if (!city) return null;

  // „Merkez“ („център“) е името на централния район на всеки турски град – не казва нищо
  const district = turkey
    ? firstOther(city, [addr.subregion, addr.city, addr.district].filter((v) => norm(v ?? '') !== 'merkez'))
    : firstOther(city, [addr.district]);
  const country = clean(addr.country);

  const second = [district, country].filter(Boolean).join(', ');
  return {
    names: { bg: city, en: city },
    detail: second ? { bg: second, en: second } : undefined,
  };
}

```

resolve.ts src/domain

- `locationFromCoords` е единственото място, което решава: в България → `placeAt` и Мюфтийство; извън → изчисление за координатите.

src/domain/resolve.ts

26 реда

```

import { formatCoords, type AppLocation } from './location';
import { isInBulgaria, placeAt, placeToLocation } from './places';

export interface PlaceName {
  names: { bg: string; en: string };
  detail?: { bg: string; en: string };
}

```

```

/**
 * От координати (GPS) към място с правилния източник на часове:
 * - в България – населеното място, в което си (виж placeAt), и календарът на Мюфтийството;
 * - извън България – изчисление за точните координати; името идва отвън
 * (обратно геокодиране от телефона), а без интернет – координатите.
 */
export function locationFromCoords(lat: number, lon: number, abroadName?: PlaceName | null): AppLocation {
  if (isInBulgaria(lat, lon)) return placeToLocation(placeAt(lat, lon));
  const coords = formatCoords(lat, lon);
  return {
    id: `gps:${lat.toFixed(3)},${lon.toFixed(3)}`,
    names: abroadName?.names ?? { bg: coords, en: coords },
    detail: abroadName?.detail,
    latitude: lat,
    longitude: lon,
    source: 'calc',
  };
}

```

calc.ts src/domain

- Седем метода за избор. Диянет остава по подразбиране.
- Над 48° ширина се прилага избраното правило за северни ширини. При полярен ден (Тромсьо през юни) – `AqrabYaum` : часовете от най-близкия ден, в който слънцето залязва.
- **Полярна нощ и полюсите:** там adhan дава невалидни часове (Аср преди Зухр в Тромсьо през декември, „NaN“ на полюса). `isValidDay` проверява, че всички часове ги има и са по ред; ако не са, `computeCalcDay` смята за същата дължина на 65° ширина (най-близките места с нормален ден и нощ, aqrab al-bilad), а ако и там не стане – още по-близо до екватора. В нормалните дни се ползва истинската ширина.

src/domain/calc.ts

120 реда

```

import {
  CalculationMethod,
  Coordinates,
  HighLatitudeRule,
  Madhab,
  PolarCircleResolution,
  PrayerTimes,
} from 'adhan';

import type { AppLocation } from './location';
import { PRAYER_IDS, type PrayerTime } from './prayers';

/** Методите, които се предлагат в настройките (извън България). */
export const CALC_METHODS = [
  'Turkey',
  'MuslimWorldLeague',
  'MoonsightingCommittee',
  'Egyptian',
  'Karachi',
  'UmmAlQura',
  'NorthAmerica',
] as const;
export type CalculationMethodId = (typeof CALC_METHODS)[number];

export type AsrMadhab = 'shafi' | 'hanafi';

```

```

/** Правило за северни ширини – как се смятат Фаджр и Иша, когато няма истински здрач. */
export const HIGH_LAT_RULES = ['seventhofthenight', 'middleofthenight', 'twilightangle'] as const;
export type HighLatRuleId = (typeof HIGH_LAT_RULES)[number];

/** Над тази ширина (на север или юг) настройката „северни ширини“ има значение и се показва. */
export const HIGH_LATITUDE = 48;

export function isHighLatitude(lat: number): boolean {
  return Math.abs(lat) > HIGH_LATITUDE;
}

/**
 * Над тази ширина има полярен ден и полярна нощ (Тромсьо, Норилск, Свалбард, полюсите).
 * В дните, в които изчислението за истинската ширина не става (няма изгрев/залез,
 * разбъркан ред), часовете са за същата дължина, но на 65° – най-близките места
 * с нормален ден и нощ („най-близката страна“, aqrab al-bilad).
 */
export const POLAR_LATITUDE = 65;

export function isPolar(lat: number): boolean {
  return Math.abs(lat) > POLAR_LATITUDE;
}

/**
 * Полярен ден или полярна нощ – слънцето този ден не изгрява или не залязва.
 * Деклинацията на слънцето е приблизителна ( $\pm 0,5^\circ$ ), затова има запас от  $1,5^\circ$ :
 * в граничните дни също се минава на 65°, вместо да се търсят часове от други дни
 * (това търсене в adhan е бавно – месец на Свалбард отнемаше ~0,5 сек. на компютър).
 */
export function isPolarDayOrNight(date: Date, lat: number): boolean {
  const start = new Date(date.getFullYear(), 0, 1);
  const dayOfYear = Math.round((date.getTime() - start.getTime()) / 86_400_000) + 1;
  const decl = 23.44 * Math.sin(((2 * Math.PI) / 365) * (284 + dayOfYear));
  return Math.abs(lat) > 90 - Math.abs(decl) - 1.5;
}

/** Всички часове са истински дати и вървят по ред: Фаджр < Изгрев < Зухр < Аср < Магриб < Иша. */
export function isValidDay(times: PrayerTime[]): boolean {
  return times.every(
    (p, i) => !isNaN(p.time.getTime()) && (i === 0 || p.time.getTime() > times[i - 1].time.getTime()),
  );
}

/**
 * Астрономическо изчисление с adhan – за места извън България.
 * Методът „Turkey“ (Диянет) вече съдържа предпазните минути (темкин) на Диянет.
 * Сравнено с календара на Мюфтийството за София, средната разлика е ~0,
 * но в отделни дни стига 2–4 мин. (виж docs/PLAN.md, етап 2).
 */
export function computeCalcDay(
  date: Date,
  location: AppLocation,
  method: CalculationMethodId,
  madhab: AsrMadhab,
  highLatRule: HighLatRuleId = 'seventhofthenight',
): PrayerTime[] {
  const lat = location.latitude;
  const sign = lat < 0 ? -1 : 1;
  // Полярен ден/нощ → направо 65°. Иначе първо истинската ширина; ако денят излезе
  // невалиден (липсващ час или разбъркан ред) – 65°, а ако и там не стане – още по-близо до екватора.
  let tryLat = isPolar(lat) && isPolarDayOrNight(date, lat) ? sign * POLAR_LATITUDE : lat;

```

```

for (;;) {
    const times = calcAt(date, tryLat, location.longitude, method, madhab, highLatRule);
    if (isValidDay(times) || Math.abs(tryLat) <= HIGH_LATITUDE) return times;
    tryLat = sign * Math.min(Math.abs(tryLat) - 1, POLAR_LATITUDE);
}
}

function calcAt(
    date: Date,
    latitude: number,
    longitude: number,
    method: CalculationMethodId,
    madhab: AsrMadhab,
    highLatRule: HighLatRuleId,
): PrayerTime[] {
    const params = CalculationMethod[method]();
    params.madhab = madhab === 'hanafi' ? Madhab.Hanafi : Madhab.Shafi;
    if (isHighLatitude(latitude)) {
        params.highLatitudeRule =
            highLatRule === 'middleofthenight'
                ? HighLatitudeRule.MiddleOfTheNight
                : highLatRule === 'twilightangle'
                    ? HighLatitudeRule.TwilightAngle
                    : HighLatitudeRule.SeventhOfTheNight;
        // резерва за граничните дни около полярния кръг – най-близкият ден с нормални часове
        params.polarCircleResolution = PolarCircleResolution.AqrabYaum;
    }
    const coords = new Coordinates(latitude, longitude);
    // обяд, за да не може смяната на лятно/зимно време да измести датата
    const noon = new Date(date.getFullYear(), date.getMonth(), date.getDate(), 12);
    const pt = new PrayerTimes(coords, noon, params);
    return PRAYER_IDS.map((id) => ({ id, time: pt[id] }));
}

```

times.ts src/domain

- **highLatRule** се подава към изчислението. Останалото е без промяна.

src/domain/times.ts

60 реда

```

import { computeCalcDay, type AsrMadhab, type CalculationMethodId, type HighLatRuleId } from './calc';
import type { AppLocation } from './location';
import { computeMuftiDay } from './mufti';
import type { PrayerId, PrayerTime, ThreeDays } from './prayers';

export interface TimesOptions {
    location: AppLocation;
    /** Само за source 'calc'. */
    method: CalculationMethodId;
    /** Само за source 'calc'. В България Аср е винаги по календара на Мюфтийството. */
    madhab: AsrMadhab;
    /** Само за source 'calc' и ширина над 48°. */
    highLatRule?: HighLatRuleId;
    /** Ръчни корекции в минути (+/-) за всяка молитва. Настройва се в етап 3. */
    offsets?: Partial<Record<PrayerId, number>>;
}

const MINUTE = 60_000;

/** Единственото място, което решава откъде идват часовете за даден ден. */

```

```

export function computeDay(date: Date, opts: TimesOptions): PrayerTime[] {
  const { location } = opts;
  const base =
    location.source === 'mufti'
      ? computeMuftiDay(date, location.muftiShift ?? 0)
      : computeCalcDay(date, location, opts.method, opts.madhab, opts.highLatRule);

  if (!opts.offsets) return base;
  return base.map((p) => {
    const off = opts.offsets?.[p.id] ?? 0;
    return off ? { id: p.id, time: new Date(p.time.getTime() + off * MINUTE) } : p;
  });
}

const dayAt = (d: Date, offset: number) =>
  new Date(d.getFullYear(), d.getMonth(), d.getDate() + offset, 12);

export function computeThreeDays(now: Date, opts: TimesOptions): ThreeDays {
  return {
    yesterday: computeDay(dayAt(now, -1), opts),
    today: computeDay(dayAt(now, 0), opts),
    tomorrow: computeDay(dayAt(now, 1), opts),
  };
}

export interface MonthDay {
  date: Date; // обяд на съответния ден
  times: PrayerTime[];
}

/** Всички дни от месеца (month: 0 = януари). */
export function computeMonth(year: number, month: number, opts: TimesOptions): MonthDay[] {
  const days = new Date(year, month + 1, 0).getDate();
  const out: MonthDay[] = [];
  for (let d = 1; d <= days; d++) {
    const date = new Date(year, month, d, 12);
    out.push({ date, times: computeDay(date, opts) });
  }
  return out;
}

```

defaults.ts src/config

- Мястото по подразбиране вече е София-град от списъка с населени места.

src/config/defaults.ts

20 реда

```

import type { AsrMadhab, CalculationMethodId, HighLatRuleId } from '@domain/calc';
import type { AppLocation } from '@domain/location';
import { defaultPlace, placeToLocation } from '@domain/places';

/**
 * Стойности по подразбиране. Потребителят ги сменя от „Настройки“ (src/store/settings.ts).
 */

/** София-град - докато няма GPS или избрано място. */
export const DEFAULT_LOCATION: AppLocation = placeToLocation(defaultPlace());

/**
 * За места извън България (source 'calc'): метод Диянет и „първият“ Аср (Шафии),

```

```

* както в календарите на Диянет и на Мюфтийството. Ханафи дава Аср около час по-късно.
*/
export const DEFAULT_METHOD: CalculationMethodId = 'Turkey';
export const DEFAULT_MADHAB: AsrMadhab = 'shafi';

/** Над 48° ширина: „една седма от нощта“ – най-често ползваното правило в Европа. */
export const DEFAULT_HIGH_LAT_RULE: HighLatRuleId = 'seventhofthenight';

```

Стъпка 5

Настройки и GPS

Настройките се пазят в телефона. Всяка промяна веднага преизчислява часовете на всички екрани.

settings.ts src/store

- Пази: автоматично/ръчно място, последното място от GPS, ръчно избраното място, метод, Аср, правило за северни ширини, корекциите и корекцията на Хиджра.
- Последното място от GPS се пази, за да се покажат правилните часове веднага при следващо отваряне – дори без GPS или без разрешение.
- Ръчният избор изключва автоматичното място, иначе следващото определяне с GPS би го „презаписало“.
- `selectLocation` – мястото, за което се показват часовете в момента. Ако няма нищо – София.

src/store/settings.ts

102 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,
  DEFAULT_METHOD,
} from '@config/defaults';
import type { AsrMadhab, CalculationMethodId, HighLatRuleId } from '@domain/calc';
import type { AppLocation } from '@domain/location';
import type { PrayerId } from '@domain/prayers';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни

const NO_OFFSETS: Record<PrayerId, number> = {
  fajr: 0,
  sunrise: 0,
  dhuhr: 0,
  asr: 0,
  maghrib: 0,
  isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
  /** true – мястото идва от GPS; false – избрано ръчно. */

```

```

autoLocation: boolean;
/** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
gpsLocation: AppLocation | null;
/** Ръчно избраното място. */
manualLocation: AppLocation | null;

method: CalculationMethodId;
madhab: AsrMadhab;
highLatRule: HighLatRuleId;
offsets: Record<PrayerId, number>;
hijriAdjust: number;

setAutoLocation: (on: boolean) => void;
setGpsLocation: (loc: AppLocation) => void;
chooseLocation: (loc: AppLocation) => void;
setMethod: (m: CalculationMethodId) => void;
setMadhab: (m: AsrMadhab) => void;
setHighLatRule: (r: HighLatRuleId) => void;
changeOffset: (id: PrayerId, delta: number) => void;
resetOffsets: () => void;
changeHijriAdjust: (delta: number) => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
      hijriAdjust: 0,

      setAutoLocation: (on) => set({ autoLocation: on }),
      setGpsLocation: (loc) => set({ gpsLocation: loc }),
      // ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
      chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
      setMethod: (method) => set({ method }),
      setMadhab: (madhab) => set({ madhab }),
      setHighLatRule: (highLatRule) => set({ highLatRule }),
      changeOffset: (id, delta) =>
        set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
      resetOffsets: () => set({ offsets: NO_OFFSETS }),
      changeHijriAdjust: (delta) =>
        set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
    }),
    {
      name: 'ezan.settings',
      version: 1,
      storage: createJSONStorage(() => AsyncStorage),
      partialize: ({ autoLocation, gpsLocation, manualLocation, method, madhab, highLatRule, offsets,
        hijriAdjust }) => ({
        autoLocation,
        gpsLocation,
        manualLocation,
        method,
        madhab,
        highLatRule,
        offsets,

```

```

        hijriAdjust,
      }},
    },
  ),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {
  const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
  return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

```

location.ts src/services

- Иска разрешение само ако още не е отказано завинаги. Точност ~100 м („Balanced“) – достатъчна, за да се различат съседни села, и пести батерия.
- Ако текущото определяне не стане за 15 секунди, взима последната известна позиция.
- Извън България пита телефона за името на мястото (изисква интернет) и го подрежда с `nameFromAddress`. Без връзка показва координатите.
- Две едновременни извиквания се обединяват в едно.

src/services/location.ts

93 реда

```

import * as Location from 'expo-location';
import { create } from 'zustand';

import { nameFromAddress } from '@/domain/abroadName';
import { isInBulgaria } from '@/domain/places';
import { locationFromCoords, type PlaceName } from '@/domain/resolve';
import { useSettings } from '@/store/settings';

/**
 * Определяне на мястото с GPS. Само докато приложението е отворено –
 * без фонов достъп до местоположението.
 */
export type LocationStatus = 'idle' | 'locating' | 'ok' | 'denied' | 'unavailable';

export const useLocationStatus = create<{ status: LocationStatus; lastFix: number }>(() => ({
  status: 'idle',
  lastFix: 0,
}));

const setStatus = (status: LocationStatus) => useLocationStatus.setState({ status });

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T> {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => reject(new Error('timeout')), ms);
    promise.then(
      (v) => {
        clearTimeout(timer);
        resolve(v);
      },
      (e) => {
        clearTimeout(timer);
        reject(e);
      },
    );
  });
}

```

```

}

/**
 * Името на мястото извън България от телефона (изисква интернет): градът,
 * а под него районът и държавата (виж nameFromAddress). Без връзка - null.
 */
async function nameAbroad(latitude: number, longitude: number): Promise<PlaceName | null> {
  try {
    const [addr] = await withTimeout(Location.reverseGeocodeAsync({ latitude, longitude }), 8000);
    return nameFromAddress(addr);
  } catch {
    return null;
  }
}

let running: Promise<void> | null = null;

/** Взима текущото местоположение и го записва като мястото от GPS. Паралелни извиквания се обединяват. */
export function refreshLocation(): Promise<void> {
  if (!running) {
    running = doRefresh().finally(() => {
      running = null;
    });
  }
  return running;
}

async function doRefresh(): Promise<void> {
  setStatus('locating');
  try {
    let perm = await Location.getForegroundPermissionsAsync();
    if (perm.status !== 'granted' && perm.canAskAgain) {
      perm = await Location.requestForegroundPermissionsAsync();
    }
    if (perm.status !== 'granted') {
      setStatus('denied');
      return;
    }
  }

  // точност ~100 м стига, за да се различат съседни села; последната известна позиция - резерва
  const position = await withTimeout(
    Location.getCurrentPositionAsync({ accuracy: Location.Accuracy.Balanced }),
    15000,
  ).catch(() => Location.getLastKnownPositionAsync());

  if (!position) {
    setStatus('unavailable');
    return;
  }

  const { latitude, longitude } = position.coords;
  const abroad = isInBulgaria(latitude, longitude) ? null : await nameAbroad(latitude, longitude);
  useSettings.getState().setGpsLocation(locationFromCoords(latitude, longitude, abroad));
  useLocationStatus.setState({ status: 'ok', lastFix: Date.now() });
} catch {
  setStatus('unavailable');
}
}

```

useLocationUpdater.ts src/hooks

- Пуска се веднъж в корена на приложението. При автоматично място: при стартиране и при връщане в приложението след 30+ минути.
- Изчаква запазените настройки да се заредят, за да не поиска GPS, ако потребителят е избрал място ръчно.

src/hooks/useLocationUpdater.ts

30 реда

```
import { useEffect, useState } from 'react';
import { AppState } from 'react-native';

import { refreshLocation, useLocationStatus } from '@services/location';
import { useSettings } from '@store/settings';

const REFRESH_AFTER_MS = 30 * 60_000; // при връщане в приложението – ако последното определяне е по-старо от 30 мин.

/**
 * Пуска се веднъж в корена на приложението. При автоматично местоположение:
 * определя мястото при стартиране и при връщане в приложението след 30+ минути.
 */
export function useLocationUpdater() {
  // изчаква запазените настройки да се заредят, иначе би поискал GPS, преди да знае,
  // че потребителят е избрал място ръчно
  const [hydrated, setHydrated] = useState(() => useSettings.persist.hasHydrated());
  useEffect(() => useSettings.persist.onFinishHydration(() => setHydrated(true)), []);

  const auto = useSettings((s) => s.autoLocation);

  useEffect(() => {
    if (!hydrated || !auto) return;
    refreshLocation();
    const sub = AppState.addEventListener('change', (state) => {
      const { lastFix } = useLocationStatus.getState();
      if (state === 'active' && Date.now() - lastFix > REFRESH_AFTER_MS) refreshLocation();
    });
    return () => sub.remove();
  }, [hydrated, auto]);
}
```

useTimesOptions.ts src/hooks

- Събира всичко, от което зависят часовете. „Днес“ и „Месец“ го ползват еднакво, затова винаги показват едни и същи часове.
- `key` е „отпечатък“ на настройките – по него се решава кога да се преизчисли.

src/hooks/useTimesOptions.ts

39 реда

```
import { useMemo } from 'react';
import { useShallow } from 'zustand/react/shallow';

import type { TimesOptions } from '@domain/times';
import { selectLocation, useSettings } from '@store/settings';

/**
 * Всичко, от което зависят часовете, събрано от настройките.
 * Ползва се от „Днес“, „Месец“ (и по-късно от нотификациите), затова всички
```

```

* екрани винаги показват едни и същи часове.
*/
export function useTimesOptions(): { options: TimesOptions; key: string } {
  const { location, method, madhab, highLatRule, offsets } = useSettings(
    useShallow((s) => ({
      location: selectLocation(s),
      method: s.method,
      madhab: s.madhab,
      highLatRule: s.highLatRule,
      offsets: s.offsets,
    })),
  );

  return useMemo(() => {
    const options: TimesOptions = { location, method, madhab, highLatRule, offsets };
    // „отпечатък“ на настройките – по него се преизчисляват часовете
    const key = JSON.stringify([
      location.id,
      location.latitude,
      location.longitude,
      location.source,
      location.muftiShift,
      method,
      madhab,
      highLatRule,
      offsets,
    ]);
    return { options, key };
  }, [location, method, madhab, highLatRule, offsets]);
}

```

usePrayerSchedule.ts src/hooks

- Преизчислява при смяна на деня **или** на настройките.

src/hooks/usePrayerSchedule.ts

20 реда

```

import { useMemo } from 'react';

import { getSchedule, type Schedule } from '@domain/prayers';
import { computeThreeDays } from '@domain/times';

import { useTimesOptions } from './useTimesOptions';

/**
 * Часовете се преизчисляват само когато се смени денят или настройките
 * (място, метод, корекции), а текущата/следващата молитва – на всяка секунда (евтино е).
 */
export function usePrayerSchedule(now: Date): Schedule {
  const { options, key } = useTimesOptions();
  const dayKey = `${now.getFullYear()}-${now.getMonth()}-${now.getDate()}`;

  // eslint-disable-next-line react-hooks/exhaustive-deps
  const days = useMemo(() => computeThreeDays(now, options), [dayKey, key]);

  return getSchedule(now, days);
}

```

Стъпка 6

Екраните

„Настройки“ по тоскир-а, нов модален екран за избор на място и промени в „Днес“, „Месец“ и корена.

controls.tsx src/components/settings

- Гравивните елементи: група с етикет, ред, ред със стрелка, превключвател, сегментиран избор (Шафии/Ханафи), „– стойност +“ и списък с радио бутони.

src/components/settings/controls.tsx

250 реда

```
import type { ReactNode } from 'react';
import { Pressable, StyleSheet, Switch, Text, View } from 'react-native';

import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

import { ChevronIcon } from '../icons';

/* Гравивните елементи на екрана „Настройки“ – по тоскир-а: група с етикет и полупрозрачна карта. */

export function Group({ label, children, note }: { label: string; children: ReactNode; note?: string }) {
  return (
    <View style={styles.group}>
      <Text style={styles.groupLabel}>{label.toUpperCase()}</Text>
      <View style={styles.card}>{children}</View>
      {note ? <Text style={styles.note}>{note}</Text> : null}
    </View>
  );
}

export function Row({ children, first }: { children: ReactNode; first?: boolean }) {
  return <View style={[styles.row, !first && styles.rowBorder]}>{children}</View>;
}

export function NavRow({
  label,
  value,
  onPress,
  first,
}: {
  label: string;
  value?: string;
  onPress: () => void;
  first?: boolean;
}) {
  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      style={({ pressed }) => [styles.row, !first && styles.rowBorder, pressed && styles.pressed]}
    >
      <Text style={styles.label}>{label}</Text>
      {value ? (
        <Text style={styles.value} numberOfLines={1}>
          {value}
        </Text>
      ) : null}
      <ChevronIcon direction="right" size={16} color={colors.muted} />
    </Pressable>
  );
}
```

```

    );
}

export function SwitchRow({
  label,
  value,
  onChange,
  first,
}: {
  label: string;
  value: boolean;
  onChange: (v: boolean) => void;
  first?: boolean;
}) {
  return (
    <Row first={first}>
      <Text style={styles.label}>{label}</Text>
      <Switch
        value={value}
        onValueChange={onChange}
        trackColor={{ false: 'rgba(255,255,255,0.18)', true: colors.gold }}
        thumbColor="#FFFFFF"
        accessibilityLabel={label}
      />
    </Row>
  );
}

export function Segmented<T extends string>({
  options,
  value,
  onChange,
}: {
  options: { value: T; label: string }[];
  value: T;
  onChange: (v: T) => void;
}) {
  return (
    <View style={styles.seg} accessibilityRole="radiogroup">
      {options.map((o) => {
        const on = o.value === value;
        return (
          <Pressable
            key={o.value}
            onPress={() => onChange(o.value)}
            accessibilityRole="radio"
            accessibilityState={{ checked: on }}
            style={[styles.segBtn, on && styles.segOn]}
          >
            <Text style={[styles.segText, on && styles.segTextOn]}>{o.label}</Text>
          </Pressable>
        );
      })}
    </View>
  );
}

/** Ред с "- стойност +". */
export function StepperRow({
  label,
  valueText,

```

```

onMinus,
onPlus,
minusLabel,
plusLabel,
highlighted,
first,
}: {
  label: string;
  valueText: string;
  onMinus: () => void;
  onPlus: () => void;
  minusLabel: string;
  plusLabel: string;
  highlighted?: boolean;
  first?: boolean;
}) {
  return (
    <Row first={first}>
      <Text style={styles.label}>{label}</Text>
      <View style={styles.stepper}>
        <Pressable onPress={onMinus} hitSlop={6} style={styles.stepBtn} accessibilityRole="button"
accessibilityLabel={minusLabel}>
          <Text style={styles.stepSign}>-</Text>
        </Pressable>
        <Text style={[styles.stepValue, highlighted && styles.gold]}>{valueText}</Text>
        <Pressable onPress={onPlus} hitSlop={6} style={styles.stepBtn} accessibilityRole="button"
accessibilityLabel={plusLabel}>
          <Text style={styles.stepSign}>+</Text>
        </Pressable>
      </View>
    </Row>
  );
}

/** Списък с избор на една стойност (радио бутони) – за метод и правило за северни ширини. */
export function Choice<T extends string>({
  options,
  value,
  onChange,
}): {
  options: { value: T; label: string; desc?: string }[];
  value: T;
  onChange: (v: T) => void;
}) {
  return (
    <View accessibilityRole="radiogroup">
      {options.map((o, i) => {
        const on = o.value === value;
        return (
          <Pressable
            key={o.value}
            onPress={() => onChange(o.value)}
            accessibilityRole="radio"
            accessibilityState={{ checked: on }}
            style={({ pressed }) => [styles.row, i > 0 && styles.rowBorder, pressed && styles.pressed]}
          >
            <View style={[styles.radio, on && styles.radioOn]}>{on && <View style={styles.radioDot} />}
          </View>
          <View style={styles.choiceText}>
            <Text style={[styles.label, on && styles.labelOn]}>{o.label}</Text>
            {o.desc ? <Text style={styles.desc}>{o.desc}</Text> : null}
          </View>
        )
      })}
    </View>
  );
}

```

```

        </View>
      </Pressable>
    );
  }}
</View>
);
}

export const settingsStyles = StyleSheet.create({
  label: { flex: 1, fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
  value: { fontFamily: fonts.medium, fontSize: 13.5, color: colors.muted, maxWidth: '55%' },
});

const styles = StyleSheet.create({
  group: { gap: 7 },
  groupLabel: {
    fontFamily: fonts.bold,
    fontSize: 11,
    letterSpacing: 1.1,
    color: colors.muted,
    paddingLeft: 6,
  },
  card: {
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.06)',
    borderRadius: 18,
    overflow: 'hidden',
  },
  note: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted, paddingHorizontal:
6 },
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    minHeight: 50,
    paddingHorizontal: 14,
    paddingVertical: 8,
  },
  rowBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  label: settingsStyles.label,
  labelOn: { color: colors.gold },
  value: settingsStyles.value,
  desc: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted, marginTop: 1 },
  seg: { flexDirection: 'row', padding: 3, borderRadius: 999, backgroundColor: 'rgba(255,255,255,0.08)' },
  segBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999 },
  segOn: { backgroundColor: colors.text },
  segText: { fontFamily: fonts.bold, fontSize: 13, color: colors.muted },
  segTextOn: { color: colors.base },
  stepper: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  stepBtn: {
    width: 32,
    height: 32,
    borderRadius: 16,
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.16)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  stepSign: { fontFamily: fonts.semibold, fontSize: 18, lineHeight: 20, color: colors.text },

```

```

stepValue: {
  minWidth: 62,
  textAlign: 'center',
  fontFamily: fonts.bold,
  fontSize: 13.5,
  color: colors.text,
  ...tabularNums,
},
gold: { color: colors.gold },
radio: {
  width: 20,
  height: 20,
  borderRadius: 10,
  borderWidth: 2,
  borderColor: 'rgba(255,255,255,0.3)',
  alignItems: 'center',
  justifyContent: 'center',
},
radioOn: { borderColor: colors.gold },
radioDot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.gold },
choiceText: { flex: 1 },
});

```

settings.tsx src/app/(tabs)

- **Място:** името, уточнението и източникът („Календар на Главно мюфтийство“ или „Изчисление за мястото“), „Смени мястото“, автоматично (GPS) и състоянието: определям... / няма разрешение (с бутон към настройките на телефона) / обнови.
- **Часове:** в България – само информация; извън България – метод, Аср и (над 48°) правило за северни ширини с обяснение; над 65° – бележка за полярните райони.
- **Аср** е списък с обяснение: „Шафии, Малики, Ханбали“ (сянката е равна на предмета) и „Ханафи“ (два пъти по-дълга, около час по-късно). Трите мезхеба смятат Аср еднакво.
- **Корекции:** ±1 мин. за всяка молитва (до ±15) с бутон „Нулирай“; дата по Хиджра ±1 ден (до ±2).
- **Дата по Хиджра:** бележката е „обявената от Мюфтийството“ в България и „обявената от местната джамия“ извън България.
- **Приложение:** ред „За приложението“ с версията (от `app.json` чрез `expo-constants`). Източниците вече не са в настройките.

src/app/(tabs)/settings.tsx

238 реда

```

import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useState } from 'react';
import { ActivityIndicator, Linking, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { PinIcon } from '@components/icons';
import {
  Choice,
  Group,
  NavRow,
  Row,
  StepperRow,
  SwitchRow,
  settingsStyles,

```

```

} from '@components/settings/controls';
import { useTabBarHeight } from '@components/TabBar';
import { CALC_METHODS, HIGH_LAT_RULES, isHighLatitude, isPolar } from '@domain/calc';
import { PRAYER_IDS } from '@domain/prayers';
import { useI18n } from '@/i18n';
import { refreshLocation, useLocationStatus } from '@services/location';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

export default function SettingsScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const s = useSettings();
  const location = selectLocation(s);
  const status = useLocationStatus((x) => x.status);
  const [open, setOpen] = useState<'method' | 'asr' | 'highLat' | null>(null);

  const isCalc = location.source === 'calc';
  const highLat = isCalc && isHighLatitude(location.latitude);
  const anyOffset = PRAYER_IDS.some((id) => s.offsets[id] !== 0);
  const toggle = (what: 'method' | 'asr' | 'highLat') => setOpen((cur) => (cur === what ? null : what));

  return (
    <View style={styles.root}>
      <GeometricPattern />
      {/* списъкът свързва над долната навигация, за да не се застъпва с нея */}
      <ScrollView
        style={{ marginBottom: tabBarHeight }}
        contentContainerStyle={styles.content, { paddingTop: insets.top + 8, paddingBottom: 24 }}
        showsVerticalScrollIndicator={false}
      >
        <Text style={styles.title}>{t.settings.title}</Text>

        {/* ---- Място ---- */}
        <Group label={t.settings.groupPlace}>
          <Pressable
            onPress={() => router.push('/place')}
            accessibilityRole="button"
            accessibilityLabel={` ${location.names[lang]}. ${t.settings.change}`}
            style={({ pressed }) => [styles.placeRow, pressed && styles.pressed]}
          >
            <PinIcon size={18} color={colors.gold} />
            <View style={styles.placeText}>
              <Text style={styles.placeName}>{location.names[lang]}</Text>
              {location.detail ? <Text style={styles.placeDetail}>{location.detail[lang]}</Text> : null}
              <Text style={styles.badge}>{isCalc ? t.settings.sourceCalc : t.settings.sourceMufti}</Text>
            </View>
            <Text style={styles.link}>{t.settings.change}</Text>
          </Pressable>
          <SwitchRow label={t.settings.auto} value={s.autoLocation} onChange={s.setAutoLocation} />
          <s.autoLocation && (
            <Row>
              <View style={styles.statusBox}>
                <View style={styles.statusLine}>
                  {status === 'locating' && <ActivityIndicator size="small" color={colors.gold} />}
                  <Text style={styles.statusText, status === 'locating' && styles.statusBusy}>
                    {status === 'locating'

```

```

        ? t.settings.locating
        : status === 'denied'
        ? t.settings.denied
        : status === 'unavailable'
        ? t.settings.unavailable
        : t.settings.autoHint}
    </Text>
  </View>
  {status === 'denied' ? (
    <Pressable onPress={() => Linking.openSettings()} accessibilityRole="button">
      <Text style={styles.link}>{t.settings.openSettings}</Text>
    </Pressable>
  ) : status !== 'locating' ? (
    <Pressable onPress={() => refreshLocation()} accessibilityRole="button">
      <Text style={styles.link}>{t.settings.refresh}</Text>
    </Pressable>
  ) : null}
  </View>
</Row>
)}
</Group>

{/* ---- Чacobe ---- */}
{isCalc ? (
  <Group
    label={t.settings.groupTimes}
    note={
      isPolar(location.latitude)
        ? t.settings.polarNote(Math.abs(location.latitude).toFixed(0))
        : highLat
        ? t.settings.highLatNote(Math.abs(location.latitude).toFixed(0))
        : t.settings.asrNote
    }
  >
    <NavRow first label={t.settings.method} value={t.methods[s.method]} onPress={() =>
toggle('method')} />
    {open === 'method' && (
      <Choice
        options={CALC_METHODS.map((m) => ({ value: m, label: t.methods[m] })))}
        value={s.method}
        onChange={(m) => {
          s.setMethod(m);
          setOpen(null);
        }}
      />
    )}
    <NavRow
      label={t.settings.asr}
      value={s.madhab === 'hanafi' ? t.settings.hanafi : t.settings.shafi}
      onPress={() => toggle('asr')}
    />
    {open === 'asr' && (
      <Choice
        options={[
          { value: 'shafi' as const, label: t.settings.shafi, desc: t.settings.shafiDesc },
          { value: 'hanafi' as const, label: t.settings.hanafi, desc: t.settings.hanafiDesc },
        ]}
        value={s.madhab}
        onChange={(m) => {
          s.setMadhab(m);
          setOpen(null);
        }}
      />
    )}
  </Group>
)}

```

```

    }}
  />
}}
{highLat && (
  <>
  <NavRow
    label={t.settings.highLat}
    value={t.highLatRules[s.highLatRule].name}
    onPress={() => toggle('highLat')}
  />
  {open === 'highLat' && (
    <Choice
      options={HIGH_LAT_RULES.map((r) => ({
        value: r,
        label: t.highLatRules[r].name,
        desc: t.highLatRules[r].desc,
      })))}
      value={s.highLatRule}
      onChange={(r) => {
        s.setHighLatRule(r);
        setOpen(null);
      }}
    />
  )}
  </>
)}
</Group>
) : (
  <Group label={t.settings.groupTimes} note={t.settings.sourceMuftiNote}>
    <Row first>
      <Text style={settingsStyles.label}>{t.settings.sourceMufti}</Text>
    </Row>
  </Group>
)}

/* ---- Коррекции ---- */
<Group label={t.settings.groupCorrections} note={t.settings.offsetsNote}>
  {PRAYER_IDS.map((id, i) => (
    <StepperRow
      key={id}
      first={i === 0}
      label={t.prayers[id]}
      valueText={t.settings.minutes(s.offsets[id])}
      highlighted={s.offsets[id] !== 0}
      onMinus={() => s.changeOffset(id, -1)}
      onPlus={() => s.changeOffset(id, 1)}
      minusLabel={t.settings.decrease(t.prayers[id])}
      plusLabel={t.settings.increase(t.prayers[id])}
    />
  ))}
  {anyOffset && (
    <Pressable onPress={s.resetOffsets} accessibilityRole="button" style={styles.resetRow}>
      <Text style={styles.link}>{t.settings.reset}</Text>
    </Pressable>
  )}
</Group>

<Group label={t.settings.hijri} note={isCalc ? t.settings.hijriNoteAbroad : t.settings.hijriNote}>
  <StepperRow
    first
    label={t.settings.hijri}

```

```

        valueText={t.settings.days(s.hijriAdjust)}
        highlighted={s.hijriAdjust !== 0}
        onMinus={() => s.changeHijriAdjust(-1)}
        onPlus={() => s.changeHijriAdjust(1)}
        minusLabel={t.settings.decrease(t.settings.hijri)}
        plusLabel={t.settings.increase(t.settings.hijri)}
      />
    </Group>

    {/* ---- Приложение: източниците и поверителността са в „За приложението“ ---- */}
    <Group label={t.settings.groupApp}>
      <NavRow first label={t.settings.about} value={APP_VERSION} onPress={() => router.push('/about')} />
    </Group>
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { paddingHorizontal: 16, gap: 20 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginLeft: 4 },
  placeRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12 },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  placeText: { flex: 1, gap: 2 },
  placeName: { fontFamily: fonts.extrabold, fontSize: 17, color: colors.text },
  placeDetail: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.muted },
  badge: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.gold, marginTop: 2 },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
  statusBox: { flex: 1, gap: 6 },
  statusLine: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  statusText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  statusBusy: { color: colors.text },
  resetRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
});

```

place.tsx src/app

- Модален екран: търсене, „Използвай местоположението ми (GPS)“ и списък. Без търсене показва всички 256 града.
- `useDeferredValue`: полето за писане се обновява веднага, а списъкът – когато JS е свободен. Индексът за търсене се строи веднага след отваряне на екрана.
- `close` затваря екрана само веднъж и проверява `router.canGoBack()`. Иначе повторно натискане на „Затвори“ праща второ „назад“, когато вече няма накъде – грешката „The action 'GO_BACK' was not handled by any navigator“.
- Ръчният избор е само за България – извън България мястото идва от GPS.

src/app/place.tsx

162 реда

```

import { router } from 'expo-router';
import { useDeferredValue, useEffect, useMemo, useRef, useState } from 'react';

```

```

import { ActivityIndicator, FlatList, Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { PinIcon } from '@components/icons';
import {
  allCities,
  isSearchIndexReady,
  placeToLocation,
  searchPlaces,
  warmSearchIndex,
  type Place,
} from '@domain/places';
import { useI18n } from '@i18n';
import { refreshLocation } from '@services/location';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

/** Избор на населено място в България с търсене (модален екран). */
export default function PlaceScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const chooseLocation = useSettings((s) => s.chooseLocation);
  const setAutoLocation = useSettings((s) => s.setAutoLocation);
  const [query, setQuery] = useState('');
  // Полето за писане се обновява веднага, а списъкът – когато JS е свободен.
  const deferredQuery = useDeferredValue(query);
  const searching = deferredQuery.trim().length >= 2;
  const results = useMemo(() => (searching ? searchPlaces(deferredQuery) : allCities()), [searching, deferredQuery]);

  // Индексът за търсене се строи на части, докато потребителят още не е започнал да пише.
  const [indexReady, setIndexReady] = useState(isSearchIndexReady);
  useEffect(() => {
    let alive = true;
    warmSearchIndex().then(() => alive && setIndexReady(true));
    return () => {
      alive = false;
    };
  }, []);
  // „Мисля“: написаното още не е стигнало до списъка или индексът не е готов.
  const busy = query.trim().length >= 2 && (!indexReady || query !== deferredQuery);

  // Затваря екрана само веднъж: повторно натискане на „Затвори“ (или избор + „Затвори“)
  // иначе праща второ „назад“, когато вече няма накъде – грешката GO_BACK.
  const closing = useRef(false);
  const close = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/');
  };

  const pick = (p: Place) => {
    chooseLocation(placeToLocation(p));
    close();
  };

  const onUseGps = () => {
    setAutoLocation(true);
    refreshLocation();
  };

```

```

    close();
};

return (
  <View style={[styles.root, { paddingTop: insets.top + 10 }]}>
    <View style={styles.header}>
      <Text style={styles.title}>{t.place.title}</Text>
      <Pressable onPress={close} hitSlop={10} accessibilityRole="button">
        <Text style={styles.close}>{t.place.close}</Text>
      </Pressable>
    </View>

    <View>
      <TextInput
        value={query}
        onChangeText={setQuery}
        placeholder={t.place.search}
        placeholderTextColor={colors.muted}
        autoFocus
        autoCorrect={false}
        style={styles.input}
        returnKeyType="search"
        accessibilityLabel={t.place.search}
      />
      {busy && (
        <ActivityIndicator
          size="small"
          color={colors.gold}
          style={styles.inputSpinner}
          accessibilityLabel={t.busy}
        />
      )}
    </View>

    <Pressable onPress={onUseGps} style={({ pressed }) => [styles.gpsRow, pressed && styles.pressed]}
    accessibilityRole="button">
      <PinIcon size={18} color={colors.gold} />
      <Text style={styles.gpsText}>{t.place.useGps}</Text>
    </Pressable>

    <FlatList
      data={results}
      keyExtractor={(p) => String(p.index)}
      keyboardShouldPersistTaps="handled"
      contentContainerStyle={{ paddingBottom: insets.bottom + 24 }}
      ListHeaderComponent={
        !searching ? <Text style={styles.section}>{t.place.cities.toUpperCase()}</Text> : null
      }
      ListEmptyComponent={<Text style={styles.empty}>{t.place.noResults}</Text>}
      ListFooterComponent={<Text style={styles.footer}>{t.place.abroadNote}</Text>}
      renderItem={({ item }) => {
        const loc = placeToLocation(item);
        return (
          <Pressable
            onPress={() => pick(item)}
            style={({ pressed }) => [styles.item, pressed && styles.pressed]}
            accessibilityRole="button"
          >
            <Text style={styles.itemName}>{loc.names[lang]}</Text>
            <Text style={styles.itemDetail}>{loc.detail?.[lang]}</Text>
          </Pressable>
        )
      }
    />
  </View>
);

```

```

    );
  }}
  />
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base, paddingHorizontal: 16 },
  header: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 12 },
  title: { fontFamily: fonts.extrabold, fontSize: 22, color: colors.text },
  close: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
  input: {
    height: 46,
    borderRadius: 14,
    paddingHorizontal: 14,
    paddingRight: 44, // място за индикатора
    backgroundColor: 'rgba(255,255,255,0.07)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.10)',
    color: colors.text,
    fontFamily: fonts.medium,
    fontSize: 15,
  },
  inputSpinner: { position: 'absolute', right: 14, top: 0, bottom: 0 },
  gpsRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 14, paddingHorizontal: 4 },
  gpsText: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  section: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted, marginTop: 4,
marginBottom: 4 },
  item: {
    paddingVertical: 11,
    paddingHorizontal: 4,
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: 'rgba(255,255,255,0.08)',
  },
  itemName: { fontFamily: fonts.bold, fontSize: 15, color: colors.text },
  itemDetail: { fontFamily: fonts.regular, fontSize: 12.5, color: colors.muted, marginTop: 1 },
  empty: { fontFamily: fonts.medium, fontSize: 14, color: colors.muted, paddingVertical: 20, textAlign:
'center' },
  footer: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted, paddingVertical: 16, textAlign:
'center' },
});

```

about.tsx src/app

- „За приложението“: знак „إذان“, име, описание и версия; източници на часовете (Мюфтийството, adhan), данни за местата (OpenStreetMap, Natural Earth) и поверителност – местоположението не излиза от телефона.
- Отваря се отдясно като обикновена страница; „< Настройки“ връща назад само веднъж (`router.canGoBack()`).

src/app/about.tsx

105 реда

```

import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useRef } from 'react';

```

```

import { Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { Group, Row } from '@components/settings/controls';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

/** „За приложението“: версия, източници на данните и поверителност. Отваря се от „Настройки“. */
export default function AboutScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const a = t.about;

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  const source = (key: keyof typeof a.sources, first?: boolean) => (
    <Row first={first}>
      <View style={styles.sourceText}>
        <Text style={styles.sourceName}>{a.sources[key].name}</Text>
        <Text style={styles.sourceDesc}>{a.sources[key].desc}</Text>
      </View>
    </Row>
  );

  return (
    <View style={styles.root}>
      <GeometricPattern />
      <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
        <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
          <ChevronIcon direction="left" size={18} color={colors.gold} />
          <Text style={styles.backText}>{a.back}</Text>
        </Pressable>
      </View>
      <ScrollView
        contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}
        showsVerticalScrollIndicator={false}
      >
        <View style={styles.hero}>
          <View style={styles.logo}>
            <Text style={styles.logoText}>أذان</Text>
          </View>
          <Text style={styles.appName}>{a.name}</Text>
          <Text style={styles.tagline}>{a.tagline}</Text>
          <Text style={styles.version}>{a.version(APP_VERSION)}</Text>
        </View>

        <Group label={a.groupTimes}>
          {source('mufti', true)}
          {source('adhan')}
        </Group>
      </ScrollView>
    </View>
  );
}

```

```

        <Group label={a.groupPlaces}>
          {source('osm', true)}
          {source('ne')}
        </Group>
        <Group label={a.groupPrivacy}>
          <Row first>
            <Text style={styles.privacy}>{a.privacy}</Text>
          </Row>
        </Group>
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: { paddingHorizontal: 16, paddingBottom: 4 },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, alignSelf: 'flex-start', paddingVertical:
6 },
  backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 20 },
  hero: { alignItems: 'center', paddingVertical: 12, gap: 4 },
  logo: {
    width: 84,
    height: 84,
    borderRadius: 24,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: 'rgba(212,168,87,0.12)',
    borderWidth: 1,
    borderColor: 'rgba(212,168,87,0.45)',
    marginBottom: 10,
  },
  logoText: { fontFamily: fonts.arabicBold, fontSize: 30, color: colors.gold },
  appName: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
  tagline: { fontFamily: fonts.medium, fontSize: 13.5, color: colors.textDim },
  version: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.muted, marginTop: 2 },
  sourceText: { flex: 1, gap: 2 },
  sourceName: { fontFamily: fonts.bold, fontSize: 14.5, color: colors.text },
  sourceDesc: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  privacy: { flex: 1, fontFamily: fonts.regular, fontSize: 13, lineHeight: 19, color: colors.textDim },
});

```

_layout.tsx src/app

- `useLocationUpdater()`, регистрация на `place` като модален екран и на `about` като обикновена страница.

src/app/_layout.tsx

84 реда

```

import { Amiri_400Regular, Amiri_700Bold } from '@expo-google-fonts/amiri';
import {
  Manrope_400Regular,
  Manrope_500Medium,
  Manrope_600SemiBold,
  Manrope_700Bold,
  Manrope_800ExtraBold,
} from '@expo-google-fonts/manrope';
import { useFonts } from 'expo-font';
import { Stack } from 'expo-router';

```

```

import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect, useState } from 'react';

import { useLocationUpdater } from '@/hooks/useLocationUpdater';
import { useAlertPrefs } from '@/store/alertPrefs';
import { useSettings } from '@/store/settings';
import { colors } from '@/theme/colors';

// Splash екранът стои, докато шрифтовете и запазените настройки се заредят -
// иначе за миг би се показвала София по подразбиране вместо твоето място.
SplashScreen.preventAutoHideAsync();

function useStoresHydrated(): boolean {
  const [done, setDone] = useState(
    () => useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated(),
  );
  useEffect(() => {
    const check = () => setDone(useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated());
    const a = useSettings.persist.onFinishHydration(check);
    const b = useAlertPrefs.persist.onFinishHydration(check);
    check();
    // предпазител: ако четенето от паметта на телефона се провали, zustand не съобщава
    // „готово“ - след 2 сек. приложението се показва с настройките по подразбиране
    const timer = setTimeout(() => setDone(true), 2000);
    return () => {
      a();
      b();
      clearTimeout(timer);
    };
  }, []);
  return done;
}

export default function RootLayout() {
  const [loaded, error] = useFonts({
    Manrope_400Regular,
    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
    Amiri_400Regular,
    Amiri_700Bold,
  });

  const hydrated = useStoresHydrated();
  const ready = (loaded || !!error) && hydrated;

  useEffect(() => {
    if (ready) SplashScreen.hideAsync();
  }, [ready]);

  // GPS при стартиране и при връщане в приложението (ако мястото е автоматично)
  useLocationUpdater();

  if (!ready) return null;

  return (
    <>
      <StatusBar style="light" />
      <Stack

```

```

    screenOptions={{
      headerShown: false,
      contentStyle: { backgroundColor: colors.base },
    }}
  >
  <Stack.Screen name="(tabs)" />
  { /* изборът на място се отваря отдолу нагоре като модален екран */ }
  <Stack.Screen name="place" options={{ presentation: 'modal' }} />
  <Stack.Screen name="about" />
</Stack>
</>
);
}

```

index.tsx src/app/(tabs)

- Името на мястото идва от настройките; тап върху него отваря избора на място; датата по Хиджра е с корекцията.

src/app/(tabs)/index.tsx

73 реда

```

import { router } from 'expo-router';
import { useCallback, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NextPrayerHero } from '@components/NextPrayerHero';
import { PhaseBackground } from '@components/PhaseBackground';
import { PrayerList } from '@components/PrayerList';
import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { TopBar } from '@components/TopBar';
import { formatGregorianShort } from '@domain/format';
import { formatHijri } from '@domain/hijri';
import type { PrayerId } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n } from '@i18n';
import { useLocationStatus } from '@services/location';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, useSettings } from '@store/settings';

export default function TodayScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const tabBarHeight = useTabBarHeight();

  const now = useNow();
  const schedule = usePrayerSchedule(now);
  const location = useSettings(selectLocation);
  const hijriAdjust = useSettings((s) => s.hijriAdjust);
  const autoLocation = useSettings((s) => s.autoLocation);
  const hasGpsLocation = useSettings((s) => s.gpsLocation !== null);
  const locating = useLocationStatus((s) => s.status === 'locating') && autoLocation;

  const cycle = useAlertPrefs((s) => s.cycle);
  const [toast, setToast] = useState<{ id: number; text: string } | null>(null);

  const onBellPress = useCallback(
    (id: PrayerId) => {

```

```

    const mode = cycle(id);
    setToast({ id: Date.now(), text: t.toast(t.prayers[id], t.alert[mode]) });
  },
  [cycle, t],
);

return (
  <View style={styles.root}>
    <PhaseBackground phase={schedule.current.id} />

    <ScrollView
      contentContainerStyle={{ paddingTop: insets.top, paddingBottom: tabBarHeight + 16 }}
      showsVerticalScrollIndicator={false}
    >
      <TopBar
        city={location.names[lang]}
        gregorian={formatGregorianShort(now, t.date)}
        hijri={formatHijri(now, t.hijriMonths, hijriAdjust)}
        onCityPress={() => router.push('/place')}
        locating={locating}
        locatingFirstTime={!hasGpsLocation}
      />
      <NextPrayerHero schedule={schedule} />
      <PrayerList schedule={schedule} onBellPress={onBellPress} />
    </ScrollView>

    <Toast message={toast} bottom={tabBarHeight + 14} />
  </View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
});

```

month.tsx src/app/(tabs)

- Същите настройки като „Днес“. Извън България най-долу пише методът, напр. „Изчислено по метод Диянет (Турция)“.

src/app/(tabs)/month.tsx

155 реда

```

import { Fragment, useDeferredValue, useEffect, useMemo, useRef, useState } from 'react';
import { ActivityIndicator, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import {
  HIJRI_SEPARATOR_HEIGHT,
  HijriMonthSeparator,
  MONTH_ROW_HEIGHT,
  MonthColumns,
  MonthRow,
} from '@components/MonthTable';
import { MonthHeader } from '@components/MonthHeader';
import { useTabBarHeight } from '@components/TabBar';
import { toHijri } from '@domain/hijri';
import { computeMonth } from '@domain/times';
import { useNow } from '@hooks/useNow';
import { useTimesOptions } from '@hooks/useTimesOptions';

```

```

import { useI18n } from '@i18n';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

export default function MonthScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const now = useNow(60_000); // тук е достатъчно веднъж в минута

  const [ym, setYm] = useState(() => ({ y: now.getFullYear(), m: now.getMonth() }));
  const shiftMonth = (delta: number) => {
    setYm(({ y, m }) => {
      const d = new Date(y, m + delta, 1);
      return { y: d.getFullYear(), m: d.getMonth() };
    });
  };

  const { options, key } = useTimesOptions();
  const hijriAdjust = useSettings((s) => s.hijriAdjust);

  // Заглавието и бутоните реагират веднага, а таблицата се смята „на заден план“:
  // useDeferredValue първо рисува екрана със старата таблица (или с индикатор при първо
  // отваряне), после изчислява новия месец. Така смяната на месеца не „замръзва“.
  const request = useMemo(() => ({ y: ym.y, m: ym.m, key, hijriAdjust }), [ym.y, ym.m, key, hijriAdjust]);
  const shown = useDeferredValue(request, null);
  const pending = shown !== request;
  const table = useMemo(() => {
    if (!shown) return null;
    const days = computeMonth(shown.y, shown.m, options);
    return { days, hijri: days.map((d) => toHijri(d.date, shown.hijriAdjust)) };
    // options се сменя заедно с key
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [shown]);

  const isCurrentMonth = ym.y === now.getFullYear() && ym.m === now.getMonth();
  const shownIsCurrent = !!shown && shown.y === now.getFullYear() && shown.m === now.getMonth();
  const todayIndex = shownIsCurrent ? now.getDate() - 1 : -1;

  // „Ребиул-ахир – Джемазиел-евел 1448“ (или две години, ако месецът ги пресича) –
  // смята се само за първия и последния ден, за да е веднага в заглавието
  const first = toHijri(new Date(ym.y, ym.m, 1), hijriAdjust);
  const last = toHijri(new Date(ym.y, ym.m + 1, 0), hijriAdjust);
  const hijriRange =
    first.month === last.month
      ? `${t.hijriMonths[first.month - 1]} ${first.year}`
      : first.year === last.year
        ? `${t.hijriMonths[first.month - 1]} - ${t.hijriMonths[last.month - 1]} ${last.year}`
        : `${t.hijriMonths[first.month - 1]} ${first.year} - ${t.hijriMonths[last.month - 1]} ${last.year}`;

  // При отваряне на текущия месец превърта така, че днешният ден да се вижда
  // (с 3 реда над него). Разделителите за нов месец по Хиджра също заемат място.
  const scrollRef = useRef<ScrollView>(null);
  useEffect(() => {
    if (!table) return;
    const target = todayIndex - 3;
    let y = 0;
    for (let i = 0; i < target; i++) {
      y += MONTH_ROW_HEIGHT + (table.hijri[i].day === 1 ? HIJRI_SEPARATOR_HEIGHT : 0);
    }
  }, [table]);

```

```

    scrollRef.current?.scrollTo({ y, animated: false });
  }, [table, todayIndex]);

  const location = options.location;

  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      <GeometricPattern />
      <MonthHeader
        title={` ${t.date.monthsFull[ym.m]} ${ym.y}`}
        hijriRange={hijriRange}
        city={location.names[lang]}
        showToday={!isCurrentMonth}
        onPrev={() => shiftMonth(-1)}
        onNext={() => shiftMonth(1)}
        onToday={() => setYm({ y: now.getFullYear(), m: now.getMonth() })}
      />
      { /* Картата свършва над долната навигация, за да не се застъпват редовете с нея. */ }
      <View style={[styles.card, { marginBottom: tabBarHeight }]}>
        <MonthColumns />
        {table ? (
          <ScrollView
            ref={scrollRef}
            showsVerticalScrollIndicator={false}
            contentContainerStyle={styles.list}
            style={pending && styles.stale}
          >
            {table.days.map((day, i) => (
              <Fragment key={i}>
                {table.hijri[i].day === 1 && (
                  <HijriMonthSeparator
                    label={` ${t.hijriMonths[table.hijri[i].month - 1]} ${table.hijri[i].year}`}
                  />
                )}
                <MonthRow day={day} hijriDay={table.hijri[i].day} isToday={i === todayIndex} />
              </Fragment>
            ))}
            <Text style={styles.source}>
              {location.source === 'mufti' ? t.month.sourceMufti :
t.month.sourceCalc(t.methods[options.method])}
            </Text>
          </ScrollView>
        ) : null}
        {(pending || !table) && (
          <View style={styles.busy} pointerEvents="none">
            <ActivityIndicator size="large" color={colors.gold} accessibilityLabel={t.busy} />
          </View>
        )}
      </View>
    );
  }

  const styles = StyleSheet.create({
    root: { flex: 1, backgroundColor: colors.base },
    card: {
      flex: 1,
      marginHorizontal: 10,
      paddingHorizontal: 2,
      paddingTop: 4,
      borderRadius: 22,

```

```

    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
    overflow: 'hidden',
  },
  list: { paddingBottom: 14 },
  stale: { opacity: 0.4 },
  busy: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, alignItems: 'center', justifyContent: 'center' },
  source: {
    fontFamily: fonts.regular,
    fontSize: 11,
    color: colors.muted,
    textAlign: 'center',
    marginTop: 14,
    paddingHorizontal: 16,
  },
});

```

src/i18n/strings.ts – нови текстове (същите ключове и на английски)

```

settings: {
  title: 'Настройки',
  groupPlace: 'Място',
  auto: 'Автоматично (GPS)',
  denied: 'Няма разрешение за местоположение.',
  highLatNote: (lat: string) => `На ${lat}° ширина през лятото няма истински здрач ...`,
  minutes: (n: number) => (n === 0 ? '0 мин' : `${n > 0 ? '+' : ''}${n} мин`),
  ...
},
methods: { Turkey: 'Диянет (Турция)', MuslimWorldLeague: 'Muslim World League', ... },
highLatRules: { seventhofthenight: { name: 'Една седма от нощта', desc: 'Най-често ползваното в Европа' },
... },
place: { title: 'Избери място', search: 'Търси населено място в България', ... },

```

Стъпка 7

Скорост и индикатори за зареждане

Телефонът изпълнява JavaScript няколко пъти по-бавно от компютъра (Hermes няма JIT). Затова първо махнахме бавното, а там, където чакането е истинско (GPS, интернет), приложението показва, че работи.

КАКВО	ПРЕДИ (КОМПЮТЪР)	СЛЕД	РЕШЕНИЕ
Месец на Свалбард (полярен ден)	450 ms	4 ms	полярен ден/нощ → направо 65°
Място по GPS извън България	14–19 ms	~0 ms	правоъгълник около България
Място по GPS в България	14 ms	8 ms	разстояние без тригонометрия в цикъла
Дата по Хиджра × 30 (Месец)	4 ms	0,4 ms	един Intl форматер вместо 30
Индекс за търсене (6 700 места)	56 ms наведнъж	на части по 500	екранът не блокира

- **isPolarDayOrNight** : по деклинацията на слънцето познава полярен ден/нощ и смята направо за 65°, без бавното търсене на „най-близък ден“ в adhan.

src/domain/calc.ts

120 реда

```
import {
  CalculationMethod,
  Coordinates,
  HighLatitudeRule,
  Madhab,
  PolarCircleResolution,
  PrayerTimes,
} from 'adhan';

import type { AppLocation } from './location';
import { PRAYER_IDS, type PrayerTime } from './prayers';

/** Методите, които се предлагат в настройките (извън България). */
export const CALC_METHODS = [
  'Turkey',
  'MuslimWorldLeague',
  'MoonsightingCommittee',
  'Egyptian',
  'Karachi',
  'UmmAlQura',
  'NorthAmerica',
] as const;
export type CalculationMethodId = (typeof CALC_METHODS)[number];

export type AsrMadhab = 'shafi' | 'hanafi';

/** Правило за северни ширини – как се смятат Фаджр и Иша, когато няма истински здрач. */
export const HIGH_LAT_RULES = ['seventhofthenight', 'middleofthenight', 'twilightangle'] as const;
export type HighLatRuleId = (typeof HIGH_LAT_RULES)[number];

/** Над тази ширина (на север или юг) настройката „северни ширини“ има значение и се показва. */
export const HIGH_LATITUDE = 48;

export function isHighLatitude(lat: number): boolean {
  return Math.abs(lat) > HIGH_LATITUDE;
}

/**
 * Над тази ширина има полярен ден и полярна нощ (Тромсьо, Норилск, Свалбард, полюсите).
 * В дните, в които изчислението за истинската ширина не става (няма изгрев/залез,
 * разбъркан ред), часовете са за същата дължина, но на 65° – най-близките места
 * с нормален ден и нощ („най-близката страна“, aqrab al-bilad).
 */
export const POLAR_LATITUDE = 65;

export function isPolar(lat: number): boolean {
  return Math.abs(lat) > POLAR_LATITUDE;
}

/**
 * Полярен ден или полярна нощ – слънцето този ден не изгрява или не залязва.
 * Деклинацията на слънцето е приблизителна (±0,5°), затова има запас от 1,5°:
 * в граничните дни също се минава на 65°, вместо да се търсят часове от други дни
 * (това търсене в adhan е бавно – месец на Свалбард отнемаше ~0,5 сек. на компютър).

```

```

*/
export function isPolarDayOrNight(date: Date, lat: number): boolean {
  const start = new Date(date.getFullYear(), 0, 1);
  const dayOfYear = Math.round((date.getTime() - start.getTime()) / 86_400_000) + 1;
  const decl = 23.44 * Math.sin(((2 * Math.PI) / 365) * (284 + dayOfYear));
  return Math.abs(lat) > 90 - Math.abs(decl) - 1.5;
}

/** Всички часове са истински дати и вървят по ред: Фаджр < Изгрев < Зухр < Аср < Магриб < Иша. */
export function isValidDay(times: PrayerTime[]): boolean {
  return times.every(
    (p, i) => !isNaN(p.time.getTime()) && (i === 0 || p.time.getTime() > times[i - 1].time.getTime()),
  );
}

/**
 * Астрономическо изчисление с adhan – за места извън България.
 * Методът „Turkey“ (Диянет) вече съдържа предпазните минути (темкин) на Диянет.
 * Сравнено с календара на Мюфтийството за София, средната разлика е ~0,
 * но в отделни дни стига 2-4 мин. (виж docs/PLAN.md, етап 2).
 */
export function computeCalcDay(
  date: Date,
  location: AppLocation,
  method: CalculationMethodId,
  madhab: AsrMadhab,
  highLatRule: HighLatRuleId = 'seventhofthenight',
): PrayerTime[] {
  const lat = location.latitude;
  const sign = lat < 0 ? -1 : 1;
  // Полярен ден/нощ → направо 65°. Иначе първо истинската ширина; ако денят излезе
  // невалиден (липсващ час или разбъркан ред) – 65°, а ако и там не стане – още по-близо до екватора.
  let tryLat = isPolar(lat) && isPolarDayOrNight(date, lat) ? sign * POLAR_LATITUDE : lat;
  for (;;) {
    const times = calcAt(date, tryLat, location.longitude, method, madhab, highLatRule);
    if (isValidDay(times) || Math.abs(tryLat) <= HIGH_LATITUDE) return times;
    tryLat = sign * Math.min(Math.abs(tryLat) - 1, POLAR_LATITUDE);
  }
}

function calcAt(
  date: Date,
  latitude: number,
  longitude: number,
  method: CalculationMethodId,
  madhab: AsrMadhab,
  highLatRule: HighLatRuleId,
): PrayerTime[] {
  const params = CalculationMethod[method]();
  params.madhab = madhab === 'hanafi' ? Madhab.Hanafi : Madhab.Shafi;
  if (isHighLatitude(latitude)) {
    params.highLatitudeRule =
      highLatRule === 'middleofthenight'
        ? HighLatitudeRule.MiddleOfTheNight
        : highLatRule === 'twilightangle'
        ? HighLatitudeRule.TwilightAngle
        : HighLatitudeRule.SeventhOfTheNight;
    // резерва за граничните дни около полярния кръг – най-близкият ден с нормални часове
    params.polarCircleResolution = PolarCircleResolution.AqrabYaum;
  }
  const coords = new Coordinates(latitude, longitude);

```

```
// обяд, за да не може смяната на лятно/зимно време да измести датата
const noon = new Date(date.getFullYear(), date.getMonth(), date.getDate(), 12);
const pt = new PrayerTimes(coords, noon, params);
return PRAYER_IDS.map((id) => ({ id, time: pt[id] }));
}
```

geo.ts src/domain

- **fastKm** – равноъгълно разстояние (грешка под 0,5 % до стотина км); **nearBulgaria** – правоъгълникът около България.

src/domain/geo.ts

44 реда

```
import border from '@data/bulgaria-border.json';

/**
 * Проверка „в България ли е тази точка“ – без интернет.
 * Границата е от Natural Earth 1:10m (public domain), опростена до ~200 м (~10 KB).
 */
const RING: readonly (readonly number[])[] = border.ring; // [дължина, ширина]

export function distanceKm(lat1: number, lon1: number, lat2: number, lon2: number): number {
  const rad = Math.PI / 180;
  const dLat = (lat2 - lat1) * rad;
  const dLon = (lon2 - lon1) * rad;
  const a =
    Math.sin(dLat / 2) ** 2 + Math.cos(lat1 * rad) * Math.cos(lat2 * rad) * Math.sin(dLon / 2) ** 2;
  return 6371 * 2 * Math.asin(Math.sqrt(a));
}

/**
 * Бързо разстояние за търсене на най-близкото място: равноъгълна проекция,
 * без тригонометрия в цикъла (cosLat се смята веднъж за точката).
 * На разстояния до стотина км грешката е под 0,5 % – за избор на населено място е без значение.
 */
export function fastKm(lat1: number, lon1: number, lat2: number, lon2: number, cosLat: number): number {
  const dy = lat2 - lat1;
  const dx = (lon2 - lon1) * cosLat;
  return 111.195 * Math.sqrt(dx * dx + dy * dy);
}

/**
 * Правоъгълникът около България (с ~5 км запас): извън него точката със сигурност не е в България. */
export function nearBulgaria(lat: number, lon: number): boolean {
  return lat > 41.18 && lat < 44.27 && lon > 22.3 && lon < 28.7;
}

/**
 * Класическият алгоритъм „лъч“: броим колко пъти хоризонтален лъч пресича границата. */
export function insideBorder(lat: number, lon: number): boolean {
  let inside = false;
  for (let i = 0, j = RING.length - 1; i < RING.length; j = i++) {
    const [xi, yi] = RING[i];
    const [xj, yj] = RING[j];
    const crosses = yi > lat !== yj > lat && lon < ((xj - xi) * (lat - yi)) / (yj - yi) + xi;
    if (crosses) inside = !inside;
  }
  return inside;
}
```

hijri.ts src/domain

- Един `Intl.DateTimeFormat` за цялото приложение. В Hermes създаването му е скъпо – като `LocaleCompare`.

src/domain/hijri.ts

100 реда

```
/**
 * Дата по Хиджра.
 * 1) Първо опитва календара Umm al-Qura през Intl (ако телефонът го поддържа).
 * 2) Иначе ползва табличния („кувейтски“) алгоритъм, който е чисто аритметичен.
 * И двата могат да се разминат с 1-2 дни с датата, обявена от Мюфтийството
 * (тя зависи от наблюдението на луната). Затова има `adjustDays` –
 * в етап 2 става настройка „Корекция на датата по Хиджра“.
 * Имената на месеците са в src/i18n/strings.ts (hijriMonths).
 */

export interface HijriDate {
  day: number;
  month: number; // 1..12
  year: number;
}

function julianDayNumber(year: number, month: number, day: number): number {
  const a = Math.floor((14 - month) / 12);
  const y = year + 4800 - a;
  const m = month + 12 * a - 3;
  return (
    day +
    Math.floor((153 * m + 2) / 5) +
    365 * y +
    Math.floor(y / 4) -
    Math.floor(y / 100) +
    Math.floor(y / 400) -
    32045
  );
}

// Един форматор за цялото приложение: в Hermes създаването на Intl.DateTimeFormat е скъпо
// (за „Месец“ бяха 30 нови форматора при всяко показване). null = телефонът не го поддържа.
let formatter: Intl.DateTimeFormat | null | undefined;
function umalquraFormatter(): Intl.DateTimeFormat | null {
  if (formatter === undefined) {
    try {
      formatter = new Intl.DateTimeFormat('en-u-ca-islamic-umalqura', {
        day: 'numeric',
        month: 'numeric',
        year: 'numeric',
      });
    } catch {
      formatter = null;
    }
  }
  return formatter;
}

function fromIntl(date: Date): HijriDate | null {
  const f = umalquraFormatter();
  if (!f) return null;
  try {
    const parts = f.formatToParts(date);
  }
}
```

```

const num = (type: string) => parseInt(parts.find((p) => p.type === type)?.value ?? '', 10);
const day = num('day');
const month = num('month');
const year = num('year');
// проверка, че наистина е върнат ислямски календар, а не григориански
if (!(day >= 1 && day <= 30 && month >= 1 && month <= 12 && year > 1400 && year < 1600)) {
    return null;
}
return { day, month, year };
} catch {
    return null;
}
}

/** Табличният алгоритъм (експортиран за тестовите). */
export function tabularHijri(date: Date): HijriDate {
    const jd = julianDayNumber(date.getFullYear(), date.getMonth() + 1, date.getDate());

    let l = jd - 1948440 + 10632;
    const n = Math.floor((l - 1) / 10631);
    l = l - 10631 * n + 354;
    const j =
        Math.floor((10985 - l) / 5316) * Math.floor((50 * l) / 17719) +
        Math.floor(l / 5670) * Math.floor((43 * l) / 15238);
    l =
        l -
        Math.floor((30 - j) / 15) * Math.floor((17719 * j) / 50) -
        Math.floor(j / 16) * Math.floor((15238 * j) / 43) +
        29;
    const month = Math.floor((24 * l) / 709);
    const day = l - Math.floor((709 * month) / 24);
    const year = 30 * n + j - 30;

    return { day, month, year };
}

export function toHijri(date: Date, adjustDays = 0): HijriDate {
    const d = new Date(date.getFullYear(), date.getMonth(), date.getDate() + adjustDays, 12);
    return fromIntl(d) ?? tabularHijri(d);
}

/** „17 Ребиул-ахир 1448“ или „17 Rabi al-Thani 1448“ */
export function formatHijri(date: Date, monthNames: readonly string[], adjustDays = 0): string {
    const h = toHijri(date, adjustDays);
    return `${h.day} ${monthNames[h.month - 1]} ${h.year}`;
}

```

places.ts src/domain

- `warmSearchIndex` строи индекса на части с пауза между тях и връща Promise; `isSearchIndexReady`.

src/domain/places.ts

256 реда

```

import data from '@data/places.json';

import { fastKm, insideBorder, nearBulgaria } from './geo';
import type { AppLocation } from './location';
import { muftiShiftFor } from './mufti';
import { normalizeForSearch, transliterate } from './translit';

```

```

/**
 * Всички ~6 700 населени места в България (OpenStreetMap, © OpenStreetMap contributors, ODbL).
 * Във файла: [име, ширина, дължина, индекс на област, индекс на община, вид].
 * Местностите („м. ...“), вилните зони и кварталите са махнати при подготовката
 * на файла – те не са населени места (оттам идваше „м. Детски град“ в София).
 */
export type PlaceKind = 0 | 1 | 2; // град, село, махала

export interface Place {
  index: number;
  name: string;
  lat: number;
  lon: number;
  oblast: string;
  obshtina: string;
  kind: PlaceKind;
}

type Row = [string, number, number, number, number, number];
const ROWS = data.places as Row[];

export const PLACES_SOURCE: string = data.source;
export const PLACES_COUNT = ROWS.length;

export function getPlace(index: number): Place {
  const [name, lat, lon, o, s, kind] = ROWS[index];
  return {
    index,
    name,
    lat,
    lon,
    oblast: data.oblasti[o],
    obshtina: data.obshtini[s],
    kind: kind as PlaceKind,
  };
}

/** Най-близката точка от списъка по права линия (за проверката „в България ли съм“). */
export function nearestPlace(lat: number, lon: number): { place: Place; km: number } {
  let best = 0;
  let bestKm = Infinity;
  const cosLat = Math.cos((lat * Math.PI) / 180);
  for (let i = 0; i < ROWS.length; i++) {
    const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
    if (km < bestKm) {
      best = i;
      bestKm = km;
    }
  }
  return { place: getPlace(best), km: bestKm };
}

/**
 * Приблизителен радиус (км) на населеното място около точката му в OpenStreetMap.
 * Всяко място е само една точка (центърът), а големият град е широк километри:
 * в Младост си на 9 км от центъра на София и по-близо до точката на село Герман.
 */
const CITY_RADIUS_KM: Record<string, number> = {
  София: 12,
  Пловдив: 6,
  Варна: 7,

```

```

Бургас: 6,
Русе: 5,
'Стара Загора': 4,
Плевен: 4,
Сливен: 3.5,
Добрич: 3.5,
Шумен: 3.5,
Перник: 3.5,
Хасково: 3.5,
Ямбол: 3.5,
Пазарджик: 3.5,
Благоевград: 3.5,
'Велико Търново': 3.5,
Габрово: 4,
Смолян: 4,
Кърджали: 3,
Кюстендил: 3,
Враца: 3,
Видин: 3,
Монтана: 3,
Ловеч: 3,
Силистра: 3,
Търговище: 3,
Разград: 3,
Казанлък: 3,
Асеновград: 3,
Дупница: 3,
'Горна Оряховица': 3,
Димитровград: 3,
};

const TOWN_RADIUS_KM = 2;
const VILLAGE_RADIUS_KM = 1;
const HAMLET_RADIUS_KM = 0.4;

function radiusKm(row: Row): number {
  if (row[5] === 0) return CITY_RADIUS_KM[row[0]] ?? TOWN_RADIUS_KM;
  return row[5] === 1 ? VILLAGE_RADIUS_KM : HAMLET_RADIUS_KM;
}

/**
 * В кое населено място съм по GPS.
 * 1. Ако съм в радиуса на едно или повече места – мястото, спрямо което съм
 * „най-навътре“ (разстояние / радиус). В Младост → София; в Кубратово,
 * което е в радиуса на София, но на 0,2 км от центъра си → Кубратово.
 * 2. Иначе – мястото с най-малко разстояние до края му (разстояние - радиус).
 */
export function placeAt(lat: number, lon: number): Place {
  let inside = -1;
  let insideRatio = Infinity;
  let outside = 0;
  let outsideGap = Infinity;
  const cosLat = Math.cos((lat * Math.PI) / 180);
  for (let i = 0; i < ROWS.length; i++) {
    const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
    const r = radiusKm(ROWS[i]);
    if (km <= r) {
      if (km / r < insideRatio) {
        inside = i;
        insideRatio = km / r;
      }
    } else if (km - r < outsideGap) {

```

```

        outside = i;
        outsideGap = km - r;
    }
}
return getPlace(inside >= 0 ? inside : outside);
}

/**
 * В България ли е точката. Границата сама не стига: опростената брегова линия
 * „изрязва“ някои плажове и крайбрежни селища (Балчик, Равда). Затова точка,
 * която е на под 1,5 км от българско населено място, също се брои за България.
 * 1,5 км е под разстоянието от Промахон (Гърция) до Кулата (2,1 км).
 */
export function isInBulgaria(lat: number, lon: number): boolean {
    if (!nearBulgaria(lat, lon)) return false; // Истанбул, Единбург... – без да обхождаме 6 700 места
    if (insideBorder(lat, lon)) return true;
    return nearestPlace(lat, lon).km < 1.5;
}

/** Името на мястото на двата езика и уточнение (община, област) – за различаване на еднакви имена. */
export function placeToLocation(place: Place): AppLocation {
    const cityLike = place.kind === 0;
    return {
        id: `bg-${place.index}`,
        names: { bg: place.name, en: transliterate(place.name) },
        detail: {
            bg: cityLike ? `обл. ${place.oblast}` : `общ. ${place.obshtina}, обл. ${place.oblast}`,
            en: cityLike
                ? `${transliterate(place.oblast)} Province`
                : `${transliterate(place.obshtina)}, ${transliterate(place.oblast)} Province`,
        },
        latitude: place.lat,
        longitude: place.lon,
        source: 'mufti',
        muftiShift: muftiShiftFor(place.lat, place.lon),
    };
}

/** София-град от списъка – мястото по подразбиране, докато няма GPS или избор. */
export function defaultPlace(): Place {
    const i = ROWS.findIndex((r) => r[0] === 'София' && r[5] === 0);
    return getPlace(i);
}

// Индексът за търсене: 6 700 транслитерации – на компютъра ~50 ms, на телефона
// няколкостотин ms. Строи се на части, за да не блокира екрана (виж warmSearchIndex).
const searchIndex: { bg: string; lat: string }[] = [];
const CHUNK = 500;

function buildMore(count: number): void {
    const end = Math.min(ROWS.length, searchIndex.length + count);
    for (let i = searchIndex.length; i < end; i++) {
        searchIndex.push({
            bg: normalizeForSearch(ROWS[i][0]),
            lat: normalizeForSearch(transliterate(ROWS[i][0])),
        });
    }
}

export function isSearchIndexReady(): boolean {
    return searchIndex.length === ROWS.length;
}

```

```

}

function index() {
  if (!isSearchIndexReady()) buildMore(ROWS.length); // търсене преди края - довършва наведнъж
  return searchIndex;
}

let warming: Promise<void> | null = null;
/**
 * Строи индекса на части от по 500 места, с пауза между тях - екранът и клавиатурата
 * остават живи. Вика се при отваряне на екрана за търсене; обещанието се изпълнява, когато е готов.
 */
export function warmSearchIndex(): Promise<void> {
  if (isSearchIndexReady()) return Promise.resolve();
  if (!warming) {
    warming = new Promise((resolve) => {
      const step = () => {
        buildMore(CHUNK);
        if (isSearchIndexReady()) resolve();
        else setTimeout(step, 0);
      };
      setTimeout(step, 0);
    });
  }
  return warming;
}

/**
 * Подредбата по азбука е направена в places.json при подготовката на данните.
 * Тук НЕ се ползва localeCompare: в Hermes (JS машината на телефона) всяко
 * localeCompare(..., 'bg') създава нов collator и е много бавно - сортирането на
 * 875 резултата за „ch“ замразяваше екрана за няколко секунди.
 * Затова при равенство се сравнява само индексът (= азбучният ред).
 */

let cities: Place[] | null = null;
/** Всички 256 града по азбучен ред - показват се, докато няма търсене. */
export function allCities(): Place[] {
  if (!cities) {
    cities = [];
    for (let i = 0; i < ROWS.length; i++) if (ROWS[i][5] === 0) cities.push(getPlace(i));
  }
  return cities;
}

/**
 * Търсене по име на кирилица или латиница („чеп“, „cher“).
 * Подредба: точно съвпадение → започва с → съдържа; при равенство - градове
 * преди села, после по азбука.
 */
export function searchPlaces(query: string, limit = 40): Place[] {
  const q = normalizeForSearch(query);
  if (q.length < 2) return [];
  const idx = index();
  const hits: { i: number; rank: number }[] = [];
  for (let i = 0; i < idx.length; i++) {
    const { bg, lat } = idx[i];
    let rank = -1;
    if (bg === q || lat === q) rank = 0;
    else if (bg.startsWith(q) || lat.startsWith(q)) rank = 1;
    else if (bg.includes(q) || lat.includes(q)) rank = 2;
  }

```

```

    if (rank >= 0) hits.push({ i, rank: rank * 10 + ROWS[i][5] });
  }
  hits.sort((a, b) => a.rank - b.rank || a.i - b.i);
  return hits.slice(0, limit).map((h) => getPlace(h.i));
}

```

`_layout.tsx` src/app

- Splash екранът стои, докато се заредят и запазените настройки – иначе за миг се показва София вместо твоето място. Предпазител: след 2 сек. приложението се показва и при грешка в паметта.

src/app/_layout.tsx

84 реда

```

import { Amiri_400Regular, Amiri_700Bold } from '@expo-google-fonts/amiri';
import {
  Manrope_400Regular,
  Manrope_500Medium,
  Manrope_600SemiBold,
  Manrope_700Bold,
  Manrope_800ExtraBold,
} from '@expo-google-fonts/manrope';
import { useFonts } from 'expo-font';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect, useState } from 'react';

import { useLocationUpdater } from '@/hooks/useLocationUpdater';
import { useAlertPrefs } from '@/store/alertPrefs';
import { useSettings } from '@/store/settings';
import { colors } from '@/theme/colors';

// Splash екранът стои, докато шрифтовете и запазените настройки се заредят -
// иначе за миг би се показвала София по подразбиране вместо твоето място.
SplashScreen.preventAutoHideAsync();

function useStoresHydrated(): boolean {
  const [done, setDone] = useState(
    () => useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated(),
  );
  useEffect(() => {
    const check = () => setDone(useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated());
    const a = useSettings.persist.onFinishHydration(check);
    const b = useAlertPrefs.persist.onFinishHydration(check);
    check();
    // предпазител: ако четенето от паметта на телефона се провали, zustand не съобщава
    // „готово“ – след 2 сек. приложението се показва с настройките по подразбиране
    const timer = setTimeout(() => setDone(true), 2000);
    return () => {
      a();
      b();
      clearTimeout(timer);
    };
  }, []);
  return done;
}

export default function RootLayout() {
  const [loaded, error] = useFonts({
    Manrope_400Regular,

```

```

    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
    Amiri_400Regular,
    Amiri_700Bold,
  });

  const hydrated = useStoresHydrated();
  const ready = (loaded || !!error) && hydrated;

  useEffect(() => {
    if (ready) SplashScreen.hideAsync();
  }, [ready]);

  // GPS при стартиране и при връщане в приложението (ако мястото е автоматично)
  useLocationUpdater();

  if (!ready) return null;

  return (
    <>
      <StatusBar style="light" />
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: { backgroundColor: colors.base },
        }}
      >
        <Stack.Screen name="(tabs)" />
        { /* изборът на място се отваря отдолу нагоре като модален екран */ }
        <Stack.Screen name="place" options={{ presentation: 'modal' }} />
        <Stack.Screen name="about" />
      </Stack>
    </>
  );
}

```

TopBar.tsx src/components

- Докато GPS търси – въртящ се индикатор вместо иконата; при първо отваряне (още няма място) пише „Определям мястото...“.

src/components/TopBar.tsx

73 реда

```

import { ActivityIndicator, Pressable, StyleSheet, Text, View } from 'react-native';

import { useI18n } from '@/i18n';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

import { ChevronDownIcon, PinIcon } from './icons';

interface Props {
  city: string;
  gregorian: string;
  hijri: string;
  onCityPress?: () => void;
  // ** GPS определя мястото в момента – въртящ се индикатор вместо иконата. */
  locating?: boolean;
}

```

```

/** Още няма място от GPS (първо отваряне) - вместо града пише „Определям мястото...“. */
locatingFirstTime?: boolean;
}

export function TopBar({ city, gregorian, hijri, onCityPress, locating, locatingFirstTime }: Props) {
  const { t } = useI18n();
  return (
    <View style={styles.bar}>
      <Pressable
        onPress={onCityPress}
        style={({ pressed }) => [styles.city, pressed && styles.pressed]}
        accessibilityRole="button"
        accessibilityLabel={locating ? `${t.a11y.locating}. ${t.a11y.city(city)}` : t.a11y.city(city)}
        accessibilityState={{ busy: !!locating }}
      >
        {locating ? (
          <ActivityIndicator size="small" color={colors.gold} style={styles.spinner} />
        ) : (
          <PinIcon color={colors.text} />
        )}
        <Text style={styles.cityText}>{locating && locatingFirstTime ? t.locating : city}</Text>
        <ChevronDownIcon color={colors.text} />
      </Pressable>
      <View style={styles.dates}>
        <Text style={styles.gregorian}>{gregorian}</Text>
        <Text style={styles.hijri}>{hijri}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  bar: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 18,
    paddingTop: 8,
    paddingBottom: 4,
    gap: 8,
  },
  city: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 7,
    paddingLeft: 10,
    paddingRight: 12,
    borderRadius: 999,
    backgroundColor: colors.pill,
  },
  pressed: { opacity: 0.7 },
  spinner: { width: 15, height: 15, transform: [{ scale: 0.8 }] },
  cityText: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
  dates: { alignItems: 'flex-end', flexShrink: 1 },
  gregorian: { fontFamily: fonts.bold, fontSize: 12, lineHeight: 16, color: colors.text },
  hijri: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.textDim },
});

```

index.tsx src/app/(tabs)

- Подава `locating` на горната лента, когато мястото е автоматично и GPS търси.

src/app/(tabs)/index.tsx

73 реда

```
import { router } from 'expo-router';
import { useCallback, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NextPrayerHero } from '@components/NextPrayerHero';
import { PhaseBackground } from '@components/PhaseBackground';
import { PrayerList } from '@components/PrayerList';
import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { TopBar } from '@components/TopBar';
import { formatGregorianShort } from '@domain/format';
import { formatHijri } from '@domain/hijri';
import type { PrayerId } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n } from '@i18n';
import { useLocationStatus } from '@services/location';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, useSettings } from '@store/settings';

export default function TodayScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const tabBarHeight = useTabBarHeight();

  const now = useNow();
  const schedule = usePrayerSchedule(now);
  const location = useSettings(selectLocation);
  const hijriAdjust = useSettings((s) => s.hijriAdjust);
  const autoLocation = useSettings((s) => s.autoLocation);
  const hasGpsLocation = useSettings((s) => s.gpsLocation !== null);
  const locating = useLocationStatus((s) => s.status === 'locating') && autoLocation;

  const cycle = useAlertPrefs((s) => s.cycle);
  const [toast, setToast] = useState<{ id: number; text: string } | null>(null);

  const onBellPress = useCallback(
    (id: PrayerId) => {
      const mode = cycle(id);
      setToast({ id: Date.now(), text: t.toast(t.prayers[id], t.alert[mode]) });
    },
    [cycle, t],
  );

  return (
    <View style={styles.root}>
      <PhaseBackground phase={schedule.current.id} />

      <ScrollView
        contentContainerStyle={{ paddingTop: insets.top, paddingBottom: tabBarHeight + 16 }}
        showsVerticalScrollIndicator={false}
      >
        <TopBar
          city={location.names[lang]}
        />
      </ScrollView>
    </View>
  );
}
```

```

        gregorian={formatGregorianShort(now, t.date)}
        hijri={formatHijri(now, t.hijriMonths, hijriAdjust)}
        onCityPress={() => router.push('/place')}
        locating={locating}
        locatingFirstTime={!hasGpsLocation}
      />
      <NextPrayerHero schedule={schedule} />
      <PrayerList schedule={schedule} onBellPress={onBellPress} />
    </ScrollView>

    <Toast message={toast} bottom={tabBarHeight + 14} />
  </View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
});

```

settings.tsx src/app/(tabs)

- Индикатор до „Определям местоположението...“.

src/app/(tabs)/settings.tsx

238 реда

```

import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useState } from 'react';
import { ActivityIndicator, Linking, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { PinIcon } from '@components/icons';
import {
  Choice,
  Group,
  NavRow,
  Row,
  StepperRow,
  SwitchRow,
  settingsStyles,
} from '@components/settings/controls';
import { useTabBarHeight } from '@components/TabBar';
import { CALC_METHODS, HIGH_LAT_RULES, isHighLatitude, isPolar } from '@domain/calc';
import { PRAYER_IDS } from '@domain/prayers';
import { useI18n } from '@i18n';
import { refreshLocation, useLocationStatus } from '@services/location';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

export default function SettingsScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const s = useSettings();
  const location = selectLocation(s);
  const status = useLocationStatus((x) => x.status);

```

```

const [open, setOpen] = useState<'method' | 'asr' | 'highLat' | null>(null);

const isCalc = location.source === 'calc';
const highLat = isCalc && isHighLatitude(location.latitude);
const anyOffset = PRAYER_IDS.some((id) => s.offsets[id] !== 0);
const toggle = (what: 'method' | 'asr' | 'highLat') => setOpen((cur) => (cur === what ? null : what));

return (
  <View style={styles.root}>
    <GeometricPattern />
    {/* списъкът свършва над долната навигация, за да не се застъпва с нея */}
    <ScrollView
      style={{ marginBottom: tabBarHeight }}
      contentContainerStyle={[styles.content, { paddingTop: insets.top + 8, paddingBottom: 24 }]}
      showsVerticalScrollIndicator={false}
    >
      <Text style={styles.title}>{t.settings.title}</Text>

      {/* ---- Място ---- */}
      <Group label={t.settings.groupPlace}>
        <Pressable
          onPress={() => router.push('/place')}
          accessibilityRole="button"
          accessibilityLabel={`${location.names[lang]}. ${t.settings.change}`}
          style={({ pressed }) => [styles.placeRow, pressed && styles.pressed]}
        >
          <PinIcon size={18} color={colors.gold} />
          <View style={styles.placeText}>
            <Text style={styles.placeName}>{location.names[lang]}</Text>
            {location.detail ? <Text style={styles.placeDetail}>{location.detail[lang]}</Text> : null}
            <Text style={styles.badge}>{isCalc ? t.settings.sourceCalc : t.settings.sourceMufti}</Text>
          </View>
          <Text style={styles.link}>{t.settings.change}</Text>
        </Pressable>
        <SwitchRow label={t.settings.auto} value={s.autoLocation} onChange={s.setAutoLocation} />
        {s.autoLocation && (
          <Row>
            <View style={styles.statusBox}>
              <View style={styles.statusLine}>
                {status === 'locating' && <ActivityIndicator size="small" color={colors.gold} />}
                <Text style={styles.statusText, status === 'locating' && styles.statusBusy}>
                  {status === 'locating'
                    ? t.settings.locating
                    : status === 'denied'
                    ? t.settings.denied
                    : status === 'unavailable'
                    ? t.settings.unavailable
                    : t.settings.autoHint}
                </Text>
              </View>
            </View>
            {status === 'denied' ? (
              <Pressable onPress={() => Linking.openSettings()} accessibilityRole="button">
                <Text style={styles.link}>{t.settings.openSettings}</Text>
              </Pressable>
            ) : status !== 'locating' ? (
              <Pressable onPress={() => refreshLocation()} accessibilityRole="button">
                <Text style={styles.link}>{t.settings.refresh}</Text>
              </Pressable>
            ) : null}
          </Row>
        )}
      </Group>
    </ScrollView>
  </View>
)

```

```

    })
  </Group>

  {/* ---- Yacobe ---- */}
  {isCalc ? (
    <Group
      label={t.settings.groupTimes}
      note={
        isPolar(location.latitude)
          ? t.settings.polarNote(Math.abs(location.latitude).toFixed(0))
          : highLat
          ? t.settings.highLatNote(Math.abs(location.latitude).toFixed(0))
          : t.settings.asrNote
      }
    >
    <NavRow first label={t.settings.method} value={t.methods[s.method]} onPress={() =>
toggle('method')} />
    {open === 'method' && (
      <Choice
        options={CALC_METHODS.map((m) => ({ value: m, label: t.methods[m] })))}
        value={s.method}
        onChange={(m) => {
          s.setMethod(m);
          setOpen(null);
        }}
      />
    )}
    <NavRow
      label={t.settings.asr}
      value={s.madhab === 'hanafi' ? t.settings.hanafi : t.settings.shafi}
      onPress={() => toggle('asr')}
    />
    {open === 'asr' && (
      <Choice
        options={[
          { value: 'shafi' as const, label: t.settings.shafi, desc: t.settings.shafiDesc },
          { value: 'hanafi' as const, label: t.settings.hanafi, desc: t.settings.hanafiDesc },
        ]}
        value={s.madhab}
        onChange={(m) => {
          s.setMadhab(m);
          setOpen(null);
        }}
      />
    )}
    {highLat && (
      <>
      <NavRow
        label={t.settings.highLat}
        value={t.highLatRules[s.highLatRule].name}
        onPress={() => toggle('highLat')}
      />
      {open === 'highLat' && (
        <Choice
          options={HIGH_LAT_RULES.map((r) => ({
            value: r,
            label: t.highLatRules[r].name,
            desc: t.highLatRules[r].desc,
          })))}
          value={s.highLatRule}
          onChange={(r) => {

```

```

                s.setHighLatRule(r);
                setOpen(null);
            }}
        />
    )}
</>
)}
</Group>
) : (
    <Group label={t.settings.groupTimes} note={t.settings.sourceMuftiNote}>
        <Row first>
            <Text style={settingsStyles.label}>{t.settings.sourceMufti}</Text>
        </Row>
    </Group>
)}

{ /* ---- Корекции ---- */
<Group label={t.settings.groupCorrections} note={t.settings.offsetsNote}>
    {PRAYER_IDS.map((id, i) => (
        <StepperRow
            key={id}
            first={i === 0}
            label={t.prayers[id]}
            valueText={t.settings.minutes(s.offsets[id])}
            highlighted={s.offsets[id] !== 0}
            onMinus={() => s.changeOffset(id, -1)}
            onPlus={() => s.changeOffset(id, 1)}
            minusLabel={t.settings.decrease(t.prayers[id])}
            plusLabel={t.settings.increase(t.prayers[id])}
        />
    ))}
    {anyOffset && (
        <Pressable onPress={s.resetOffsets} accessibilityRole="button" style={styles.resetRow}>
            <Text style={styles.link}>{t.settings.reset}</Text>
        </Pressable>
    )}
</Group>

<Group label={t.settings.hijri} note={isCalc ? t.settings.hijriNoteAbroad : t.settings.hijriNote}>
    <StepperRow
        first
        label={t.settings.hijri}
        valueText={t.settings.days(s.hijriAdjust)}
        highlighted={s.hijriAdjust !== 0}
        onMinus={() => s.changeHijriAdjust(-1)}
        onPlus={() => s.changeHijriAdjust(1)}
        minusLabel={t.settings.decrease(t.settings.hijri)}
        plusLabel={t.settings.increase(t.settings.hijri)}
    />
</Group>

{ /* ---- Приложение: източниците и поверителността са в „За приложението“ ---- */
<Group label={t.settings.groupApp}>
    <NavRow first label={t.settings.about} value={APP_VERSION} onPress={() => router.push('/about')}>
/
    </Group>
</ScrollView>
</View>
);
}

```

```
const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { paddingHorizontal: 16, gap: 20 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginLeft: 4 },
  placeRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12 },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  placeText: { flex: 1, gap: 2 },
  placeName: { fontFamily: fonts.extrabold, fontSize: 17, color: colors.text },
  placeDetail: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.muted },
  badge: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.gold, marginTop: 2 },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
  statusBox: { flex: 1, gap: 6 },
  statusLine: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  statusText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  statusBusy: { color: colors.text },
  resetRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
});
```

month.tsx src/app/(tabs)

- `useDeferredValue(request, null)` : заглавието и стрелките реагират веднага, таблицата се смята след това. При първо отваряне – индикатор, при смяна на месеца – старата таблица избледнява с индикатор отгоре.

src/app/(tabs)/month.tsx

155 реда

```
import { Fragment, useDeferredValue, useEffect, useMemo, useRef, useState } from 'react';
import { ActivityIndicator, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import {
  HIJRI_SEPARATOR_HEIGHT,
  HijriMonthSeparator,
  MONTH_ROW_HEIGHT,
  MonthColumns,
  MonthRow,
} from '@components/MonthTable';
import { MonthHeader } from '@components/MonthHeader';
import { useTabBarHeight } from '@components/TabBar';
import { toHijri } from '@domain/hijri';
import { computeMonth } from '@domain/times';
import { useNow } from '@hooks/useNow';
import { useTimesOptions } from '@hooks/useTimesOptions';
import { useI18n } from '@i18n';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

export default function MonthScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const now = useNow(60_000); // тук е достатъчно веднъж в минута
```

```

const [ym, setYm] = useState(() => ({ y: now.getFullYear(), m: now.getMonth() }));
const shiftMonth = (delta: number) =>
  setYm(({ y, m }) => {
    const d = new Date(y, m + delta, 1);
    return { y: d.getFullYear(), m: d.getMonth() };
  });

const { options, key } = useTimesOptions();
const hijriAdjust = useSettings((s) => s.hijriAdjust);

// Заглавието и бутоните реагират веднага, а таблицата се смята „на заден план“:
// useDeferredValue първо рисува екрана със старата таблица (или с индикатор при първо
// отваряне), после изчислява новия месец. Така смяната на месеца не „замръзва“.
const request = useMemo(() => ({ y: ym.y, m: ym.m, key, hijriAdjust }), [ym.y, ym.m, key, hijriAdjust]);
const shown = useDeferredValue(request, null);
const pending = shown !== request;
const table = useMemo(() => {
  if (!shown) return null;
  const days = computeMonth(shown.y, shown.m, options);
  return { days, hijri: days.map((d) => toHijri(d.date, shown.hijriAdjust)) };
  // options се сменя заедно с key
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [shown]);

const isCurrentMonth = ym.y === now.getFullYear() && ym.m === now.getMonth();
const shownIsCurrent = !!shown && shown.y === now.getFullYear() && shown.m === now.getMonth();
const todayIndex = shownIsCurrent ? now.getDate() - 1 : -1;

// „Ребиул-ахир – Джемазиел-евел 1448“ (или две години, ако месецът ги пресича) –
// смята се само за първия и последния ден, за да е веднага в заглавието
const first = toHijri(new Date(ym.y, ym.m, 1), hijriAdjust);
const last = toHijri(new Date(ym.y, ym.m + 1, 0), hijriAdjust);
const hijriRange =
  first.month === last.month
    ? `${t.hijriMonths[first.month - 1]} ${first.year}`
    : first.year === last.year
      ? `${t.hijriMonths[first.month - 1]} - ${t.hijriMonths[last.month - 1]} ${last.year}`
      : `${t.hijriMonths[first.month - 1]} ${first.year} - ${t.hijriMonths[last.month - 1]}
    ${last.year}`;

// При отваряне на текущия месец превърта така, че днешният ден да се вижда
// (с 3 реда над него). Разделителите за нов месец по Хиджра също заемат място.
const scrollRef = useRef<ScrollView>(null);
useEffect(() => {
  if (!table) return;
  const target = todayIndex - 3;
  let y = 0;
  for (let i = 0; i < target; i++) {
    y += MONTH_ROW_HEIGHT + (table.hijri[i].day === 1 ? HIJRI_SEPARATOR_HEIGHT : 0);
  }
  scrollRef.current?.scrollTo({ y, animated: false });
}, [table, todayIndex]);

const location = options.location;

return (
  <View style={[styles.root, { paddingTop: insets.top }]}>
    <GeometricPattern />
    <MonthHeader
      title={` ${t.date.monthsFull[ym.m]} ${ym.y} `}
    />
  </View>

```

```

    hijriRange={hijriRange}
    city={location.names[lang]}
    showToday={!isCurrentMonth}
    onPrev={() => shiftMonth(-1)}
    onNext={() => shiftMonth(1)}
    onToday={() => setYm({ y: now.getFullYear(), m: now.getMonth() })}
  />
  {/* Картата свършва над долната навигация, за да не се застъпват редовете с нея. */}
  <View style={styles.card, { marginBottom: tabBarHeight }}>
    <MonthColumns />
    {table ? (
      <ScrollView
        ref={scrollRef}
        showsVerticalScrollIndicator={false}
        contentContainerStyle={styles.list}
        style={pending && styles.stale}
      >
        {table.days.map((day, i) => (
          <Fragment key={i}>
            {table.hijri[i].day === 1 && (
              <HijriMonthSeparator
                label={`$${t.hijriMonths[table.hijri[i].month - 1]} $${table.hijri[i].year}`}
              />
            )}
            <MonthRow day={day} hijriDay={table.hijri[i].day} isToday={i === todayIndex} />
          </Fragment>
        ))}
        <Text style={styles.source}>
          {location.source === 'mufti' ? t.month.sourceMufti :
t.month.sourceCalc(t.methods[options.method])}
        </Text>
      </ScrollView>
    ) : null}
    {(pending || !table) && (
      <View style={styles.busy} pointerEvents="none">
        <ActivityIndicator size="large" color={colors.gold} accessibilityLabel={t.busy} />
      </View>
    )}
  </View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  card: {
    flex: 1,
    marginHorizontal: 10,
    paddingHorizontal: 2,
    paddingTop: 4,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
    overflow: 'hidden',
  },
  list: { paddingBottom: 14 },
  stale: { opacity: 0.4 },
  busy: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, alignItems: 'center', justifyContent:
'center' },
  source: {

```

```

    fontFamily: fonts.regular,
    fontSize: 11,
    color: colors.muted,
    textAlign: 'center',
    marginTop: 14,
    paddingHorizontal: 16,
  },
});

```

place.tsx src/app

- Индикатор в полето за търсене, докато индексът се строи или написаното още не е стигнало до списъка.

src/app/place.tsx

162 реда

```

import { router } from 'expo-router';
import { useDeferredValue, useEffect, useMemo, useRef, useState } from 'react';
import { ActivityIndicator, FlatList, Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { PinIcon } from '@components/icons';
import {
  allCities,
  isSearchIndexReady,
  placeToLocation,
  searchPlaces,
  warmSearchIndex,
  type Place,
} from '@domain/places';
import { useI18n } from '@i18n';
import { refreshLocation } from '@services/location';
import { useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

/** Избор на населено място в България с търсене (модален екран). */
export default function PlaceScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const chooseLocation = useSettings((s) => s.chooseLocation);
  const setAutoLocation = useSettings((s) => s.setAutoLocation);
  const [query, setQuery] = useState('');
  // Полето за писане се обновява веднага, а списъкът - когато JS е свободен.
  const deferredQuery = useDeferredValue(query);
  const searching = deferredQuery.trim().length >= 2;
  const results = useMemo(() => (searching ? searchPlaces(deferredQuery) : allCities()), [searching, deferredQuery]);

  // Индексът за търсене се строи на части, докато потребителят още не е започнал да пише.
  const [indexReady, setIndexReady] = useState(isSearchIndexReady);
  useEffect(() => {
    let alive = true;
    warmSearchIndex().then(() => alive && setIndexReady(true));
    return () => {
      alive = false;
    };
  }, []);
  // „Мисля“: написаното още не е стигнало до списъка или индексът не е готов.
  const busy = query.trim().length >= 2 && (!indexReady || query !== deferredQuery);

```

```

// Затваря екрана само веднъж: повторно натискане на „Затвори“ (или избор + „Затвори“)
// иначе праща второ „назад“, когато вече няма накъде – грешката GO_BACK.
const closing = useRef(false);
const close = () => {
  if (closing.current) return;
  closing.current = true;
  if (router.canGoBack()) router.back();
  else router.replace('/');
};

const pick = (p: Place) => {
  chooseLocation(placeToLocation(p));
  close();
};

const onUseGps = () => {
  setAutoLocation(true);
  refreshLocation();
  close();
};

return (
  <View style={[styles.root, { paddingTop: insets.top + 10 }]}>
    <View style={styles.header}>
      <Text style={styles.title}>{t.place.title}</Text>
      <Pressable onPress={close} hitSlop={10} accessibilityRole="button">
        <Text style={styles.close}>{t.place.close}</Text>
      </Pressable>
    </View>

    <View>
      <TextInput
        value={query}
        onChangeText={setQuery}
        placeholder={t.place.search}
        placeholderTextColor={colors.muted}
        autoFocus
        autoCorrect={false}
        style={styles.input}
        returnKeyType="search"
        accessibilityLabel={t.place.search}
      />
      {busy && (
        <ActivityIndicator
          size="small"
          color={colors.gold}
          style={styles.inputSpinner}
          accessibilityLabel={t.busy}
        />
      )}
    </View>

    <Pressable onPress={onUseGps} style={({ pressed }) => [styles.gpsRow, pressed && styles.pressed]}
    accessibilityRole="button">
      <PinIcon size={18} color={colors.gold} />
      <Text style={styles.gpsText}>{t.place.useGps}</Text>
    </Pressable>

    <FlatList
      data={results}

```

```

keyExtractor={({p}) => String(p.index)}
keyboardShouldPersistTaps="handled"
contentContainerStyle={{ paddingBottom: insets.bottom + 24 }}
ListHeaderComponent={
  !searching ? <Text style={styles.section}>{t.place.cities.toUpperCase()}</Text> : null
}
ListEmptyComponent={<Text style={styles.empty}>{t.place.noResults}</Text>}
ListFooterComponent={<Text style={styles.footer}>{t.place.abroadNote}</Text>}
renderItem={({ item }) => {
  const loc = placeToLocation(item);
  return (
    <Pressable
      onPress={() => pick(item)}
      style={({ pressed }) => [styles.item, pressed && styles.pressed]}
      accessibilityRole="button"
    >
      <Text style={styles.itemName}>{loc.names[lang]}</Text>
      <Text style={styles.itemDetail}>{loc.detail?.[lang]}</Text>
    </Pressable>
  );
}}
/>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base, paddingHorizontal: 16 },
  header: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 12 },
  title: { fontFamily: fonts.extrabold, fontSize: 22, color: colors.text },
  close: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
  input: {
    height: 46,
    borderRadius: 14,
    paddingHorizontal: 14,
    paddingRight: 44, // место за индикатора
    backgroundColor: 'rgba(255,255,255,0.07)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.10)',
    color: colors.text,
    fontFamily: fonts.medium,
    fontSize: 15,
  },
  inputSpinner: { position: 'absolute', right: 14, top: 0, bottom: 0 },
  gpsRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 14, paddingHorizontal: 4 },
  gpsText: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  section: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted, marginTop: 4,
marginBottom: 4 },
  item: {
    paddingVertical: 11,
    paddingHorizontal: 4,
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: 'rgba(255,255,255,0.08)',
  },
  itemName: { fontFamily: fonts.bold, fontSize: 15, color: colors.text },
  itemDetail: { fontFamily: fonts.regular, fontSize: 12.5, color: colors.muted, marginTop: 1 },
  empty: { fontFamily: fonts.medium, fontSize: 14, color: colors.muted, paddingVertical: 20, textAlign:
'center' },

```

```
    footer: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted, paddingVertical: 16, textAlign:
'center' },
  });
```

Бавните места, които остават: GPS (до 15 сек. на закрито) и името на града извън България (иска интернет, до 8 сек.). И двете вече се виждат като „мисли“.

Стъпка 8

Тестове

100 теста общо, 74 от тях нови.

places.test.ts src/domain/__tests__

- Граница: София, Чепинци, Кулата, Капитан Андреево, Силистра и Балчик са в България; Одрин, Промахон, Гюргево, Калафат, Истанбул и Единбург – не.
- Чепинци по GPS: името на селото (не на общината), уточнението, София –6 мин. и точните часове.
- София по GPS: при „м. Детски град“, Младост 4, Горубляне, Дружба 2, Люлин 10, Обеля, Бояна и Надежда показва „София“; Кубратово, Герман, Бусманци и Каменар (до Варна) остават села. Във файла няма местности и вилни зони.
- Търсене на кирилица и латиница, транслитерация; данните са подредени по азбука; 256-те града; „ch“ дава първо градовете, по азбука.

src/domain/__tests__/places.test.ts

171 реда

```
import { formatHM } from '../format';
import { insideBorder } from '../geo';
import data from '@data/places.json';

import { allCities, isInBulgaria, nearestPlace, placeToLocation, searchPlaces } from '../places';
import { locationFromCoords } from '../resolve';
import { computeDay } from '../times';
import { transliterate } from '../translit';

const opts = { method: 'Turkey' as const, madhab: 'shafi' as const };
const times = (lat: number, lon: number, date = new Date(2026, 8, 29, 12)) =>
  computeDay(date, { ...opts, location: locationFromCoords(lat, lon) }).map((p) => formatHM(p.time));

describe('в България ли съм', () => {
  it.each([
    ['София', 42.6977, 23.3219],
    ['Чепинци (Смолянско)', 41.4348, 24.8682],
    ['Кулата', 41.39, 23.36],
    ['Капитан Андреево', 41.72, 26.32],
    ['Силистра (на Дунава)', 44.1183, 27.26],
    ['Балчик (на брега)', 43.4049, 28.1681],
  ])('%s - да', (_, lat, lon) => {
    expect(isInBulgaria(lat, lon)).toBe(true);
  });

  it.each([
    ['Одрин', 41.6771, 26.5557],
    ['Промахон (на 2 км от Кулата)', 41.3703, 23.3706],
    ['Гюргево (срещу Русе)', 43.9037, 25.9699],
    ['Калафат (срещу Видин)', 43.99, 22.93],
    ['Истанбул', 41.0082, 28.9784],
  ])
```

```

    ['Единбург', 55.9533, -3.1883],
  ))('%s - не', (_, lat, lon) => {
    expect(isInBulgaria(lat, lon)).toBe(false);
  });

  it('границата сама изрязва някои крайбрежни места - затова има и проверка за разстояние', () => {
    expect(insideBorder(43.4049, 28.1681)).toBe(false); // Балчик извън опростената граница
    expect(isInBulgaria(43.4049, 28.1681)).toBe(true);
  });
});

describe('населено място по GPS', () => {
  it('Чепинци, Смолянско - името на селото, а не на общината', () => {
    const loc = locationFromCoords(41.4352, 24.8690);
    expect(loc.names.bg).toBe('Чепинци');
    expect(loc.names.en).toBe('Chepintsi');
    expect(loc.detail?.bg).toBe('общ. Рудозем, обл. Смолян');
    expect(loc.source).toBe('mufti');
  });

  it('Чепинци е София -6 мин. (най-близкият град от календара е Мадан)', () => {
    expect(locationFromCoords(41.4348, 24.8682).muftiShift).toBe(-6);
    expect(times(41.4348, 24.8682)).toEqual(['05:39', '07:08', '13:16', '16:35', '19:15', '20:37']);
  });

  it('двете села Чепинци се различават по община и област', () => {
    const both = searchPlaces('Чепинци').filter((p) => p.name === 'Чепинци');
    expect(both).toHaveLength(2);
    const details = both.map((p) => placeToLocation(p).detail?.bg);
    expect(details).toContain('общ. Рудозем, обл. Смолян');
    expect(details).toContain('общ. Столична, обл. София-град');
  });

  it.each([
    ['при „М. Детски град“ (грешката от телефона)', 42.6135, 23.3966],
    ['Младост 4', 42.628, 23.38],
    ['Горубляне', 42.63, 23.41],
    ['Дружба 2', 42.655, 23.41],
    ['Люлин 10', 42.72, 23.245],
    ['Обеля', 42.745, 23.25],
    ['Бояна', 42.645, 23.265],
    ['Надежда', 42.735, 23.3],
  ])('в София, %s → „София“, а не местност или село', (_, lat, lon) => {
    const loc = locationFromCoords(lat, lon);
    expect(loc.names.bg).toBe('София');
    expect(loc.detail?.bg).toBe('обл. София-град');
  });

  it.each([
    ['Кубратово (в радиуса на София)', 42.7755, 23.3625, 'Кубратово', 'общ. Столична, обл. София-град'],
    ['Герман', 42.6135, 23.4150, 'Герман', 'общ. Столична, обл. София-град'],
    ['Бусманци', 42.6770, 23.4360, 'Бусманци', 'общ. Столична, обл. София-град'],
    ['Каменар до Варна', 43.2500, 27.9110, 'Каменар', 'общ. Варна, обл. Варна'],
  ])('село до голям град си остава село: %s', (_, lat, lon, name, detail) => {
    const loc = locationFromCoords(lat, lon);
    expect(loc.names.bg).toBe(name);
    expect(loc.detail?.bg).toBe(detail);
  });

  it('в списъка няма местности, вилни зони и квартали', () => {
    expect(searchPlaces('Детски град')).toEqual([]);
  });

```

```

    expect(searchPlaces('м. ').length).toBe(0);
    expect(searchPlaces('вилна зона')).toEqual([]);
  });

  it('град от календара дава точно неговата разлика (Варна -18)', () => {
    const { place } = nearestPlace(43.2074, 27.9167);
    expect(place.name).toBe('Варна');
    expect(placeToLocation(place).muftiShift).toBe(-18);
  });
});

describe('извън България', () => {
  it('Истанбул – изчисление за координатите', () => {
    const loc = locationFromCoords(41.0082, 28.9784, { names: { bg: 'Истанбул', en: 'Istanbul' } });
    expect(loc.source).toBe('calc');
    expect(loc.names.bg).toBe('Истанбул');
  });

  it('без име (няма интернет) – показва координатите', () => {
    expect(locationFromCoords(55.9533, -3.1883).names.en).toBe('55.95°N 3.19°W');
  });

  it('Единбург през лятото: Иша и Фаджр не се сливат с „една седма от нощта“', () => {
    const t = times(55.9533, -3.1883, new Date(2026, 5, 21, 12));
    // в тестовите часовата зона е Europe/Sofia (+2 ч спрямо Лондон лятото)
    const [fajr, , , maghrib, isha] = t;
    expect(isha > maghrib).toBe(true);
    expect(fajr).not.toBe(isha);
  });
});

describe('търсене', () => {
  it('на кирилица и на латиница', () => {
    expect(searchPlaces('чепин').map((p) => p.name)).toContain('Чепинци');
    expect(searchPlaces('chepin').map((p) => p.name)).toContain('Чепинци');
  });

  it('точното съвпадение и градовете са първи', () => {
    expect(searchPlaces('Варна')[0].name).toBe('Варна');
    expect(searchPlaces('Смолян')[0].kind).toBe(0);
  });

  it('списъкът е подреден по азбука още в данните (без бавното localeCompare на телефона)', () => {
    const c = new Intl.Collator('bg');
    const names = (data.places as [string][]).map((r) => r[0]);
    for (let i = 1; i < names.length; i++) expect(c.compare(names[i - 1],
names[i])).toBeLessThanOrEqual(0);
  });

  it('градовете без търсене: 256, по азбука, от Айтос', () => {
    const list = allCities();
    expect(list).toHaveLength(256);
    expect(list[0].name).toBe('Айтос');
    expect(list.every((p) => p.kind === 0)).toBe(true);
  });

  it('„ch“ (875 съвпадения) – първо градовете, по азбука', () => {
    const r = searchPlaces('ch');
    expect(r).toHaveLength(40);
    const cityNames = r.filter((p) => p.kind === 0).map((p) => p.name);
    expect(cityNames).toEqual([...cityNames].sort(new Intl.Collator('bg').compare));
  });
});

```

```
});

it('под 2 букви не търси', () => {
  expect(searchPlaces('с')).toEqual([]);
});

describe('транслитерация', () => {
  it.each([
    ['Кърджали', 'Kardzhali'],
    ['Свищов', 'Svishtov'],
    ['Провадия', 'Provadia'],
    ['Велико Търново', 'Veliko Tarnovo'],
    ['София', 'Sofia'],
    ['Горна Оряховица', 'Gorna Oryahovitsa'],
  ])('%s → %s', (bg, en) => {
    expect(transliterate(bg)).toBe(en);
  });
});
```

abroadName.test.ts src/domain/__tests__

- Истанбул от Android и от iPhone, Одрин („Merkez“), Единбург с и без квартал, място без град, празен отговор.

src/domain/__tests__/abroadName.test.ts

52 реда

```
import { nameFromAddress } from '../abroadName';

const lines = (a: Parameters<typeof nameFromAddress>[0]) => {
  const n = nameFromAddress(a);
  return n ? [n.names.bg, n.detail?.bg ?? ''] : null;
};

describe('име на мястото извън България', () => {
  it('Истанбул (Android връща района като град) → İstanbul / Fatih, Türkiye', () => {
    expect(
      lines({ city: 'Fatih', district: 'Sultanahmet', subregion: 'Fatih', region: 'İstanbul', country: 'Türkiye', isoCountryCode: 'TR' }),
    ).toEqual(['İstanbul', 'Fatih, Türkiye']);
  });

  it('Истанбул (iOS връща града) → İstanbul / Kadıköy, Türkiye', () => {
    expect(
      lines({ city: 'Istanbul', district: 'Moda', subregion: 'Kadıköy', region: 'İstanbul', country: 'Türkiye', isoCountryCode: 'TR' }),
    ).toEqual(['İstanbul', 'Kadıköy, Türkiye']);
  });

  it('Одрин – централният район „Merkez“ не се показва → Edirne / Türkiye', () => {
    expect(
      lines({ city: 'Edirne', subregion: 'Merkez', region: 'Edirne', country: 'Türkiye', isoCountryCode: 'TR' }),
    ).toEqual(['Edirne', 'Türkiye']);
    expect(lines({ city: 'Edirne', region: 'Edirne', country: 'Türkiye', isoCountryCode: 'TR' })).toEqual([
      'Edirne',
      'Türkiye',
    ]);
  });
});
```

```

it('Единбург – районът е кварталът; „City of Edinburgh“ не се повтаря', () => {
  expect(
    lines({ city: 'Edinburgh', district: 'Leith', subregion: 'City of Edinburgh', region: 'Scotland',
country: 'United Kingdom', isoCountryCode: 'GB' }),
    ).toEqual(['Edinburgh', 'Leith, United Kingdom']);
  expect(
    lines({ city: 'Edinburgh', subregion: 'City of Edinburgh', region: 'Scotland', country: 'United
Kingdom', isoCountryCode: 'GB' }),
    ).toEqual(['Edinburgh', 'United Kingdom']);
  });

it('без град (извън населено място) → окръгът, после областта', () => {
  expect(lines({ subregion: 'Highland', region: 'Scotland', country: 'United Kingdom', isoCountryCode:
'GB' })).toEqual([
    'Highland',
    'United Kingdom',
  ]);
  expect(lines({ region: 'Scotland', country: 'United Kingdom' })).toEqual(['Scotland', 'United
Kingdom']);
  });

it('празен отговор → null (показват се координатите)', () => {
  expect(nameFromAddress(null)).toBeNull();
  expect(nameFromAddress({ country: 'Türkiye' })).toBeNull();
  });
});

```

world.test.ts src/domain/__tests__

- Всяка ширина от -90° до 90° през 5° , 6 дължини, 24 дни в годината – всички часове ги има и са по ред. Същото за всички 7 метода с Ханафи.
- Тромсьо, Норилск и Свалбард при полярна нощ и двата полюса → часовете за 65° . Тромсьо на 30 септември и полярният ден на Свалбард → истинската ширина. Анкъридж, Икалуит, Ушуая – без промяна.

src/domain/__tests__/world.test.ts

87 реда

```

import { CALC_METHODS, computeCalcDay, isPolar, isValidDay, POLAR_LATITUDE } from '../calc';
import type { AppLocation } from '../location';

const at = (lat: number, lon: number): AppLocation => ({
  id: `${lat},${lon}`,
  names: { bg: '', en: '' },
  latitude: lat,
  longitude: lon,
  source: 'calc',
});

const day = (m: number, d: number) => new Date(2026, m, d);

describe('часове за целия свят', () => {
  it('всяка ширина от Южния до Северния полюс, всеки месец – всички часове ги има и са по ред', () => {
    const bad: string[] = [];
    for (let lat = -90; lat <= 90; lat += 5) {
      for (const lon of [-150, -70, 0, 25, 90, 150]) {
        for (let m = 0; m < 12; m++) {
          for (const d of [1, 21]) {
            const t = computeCalcDay(day(m, d), at(lat, lon), 'MuslimWorldLeague', 'shafi');
            if (!isValidDay(t)) bad.push(`${lat},${lon} ${m + 1}/${d}`);
          }
        }
      }
    }
  });
});

```

```

    }
  }
}
expect(bad).toEqual([]);
});

it.each(CALC_METHODS)('метод %s, Ханафи - също без грешки по целия свят', (method) => {
  const bad: string[] = [];
  for (let lat = -90; lat <= 90; lat += 10) {
    for (const lon of [-70, 25, 150]) {
      for (let m = 0; m < 12; m += 2) {
        const t = computeCalcDay(day(m, 21), at(lat, lon), method, 'hanafi');
        if (!isValidDay(t)) bad.push(`${lat},${lon} ${m + 1}/21`);
      }
    }
  }
  expect(bad).toEqual([]);
});

it.each([
  ['Тромсьо, полярна нощ', 69.65, 18.96, day(11, 21)],
  ['Норилск, полярна нощ', 69.35, 88.2, day(11, 21)],
  ['Свалбард, полярна нощ', 78.22, 15.65, day(11, 21)],
  ['Свалбард, полярна нощ', 78.22, 15.65, day(5, 21)],
  ['Тромсьо, полярна нощ', 69.65, 18.96, day(5, 21)],
  ['Северен полюс', 89.99, 0, day(5, 21)],
  ['Южен полюс', -89.99, 0, day(5, 21)],
])('%s - няма изгрев/залез → часовете са за 65° ширина и са по ред', (_, lat, lon, date) => {
  expect(isPolar(lat)).toBe(true);
  const t = computeCalcDay(date, at(lat, lon), 'MuslimWorldLeague', 'shafi');
  expect(isValidDay(t)).toBe(true);
  const at65 = computeCalcDay(date, at(Math.sign(lat) * POLAR_LATITUDE, lon), 'MuslimWorldLeague', 'shafi');
  expect(t.map((p) => p.time.getTime()).toEqual(at65.map((p) => p.time.getTime())));
});

it('полярна нощ в нормален ден (Тромсьо, 30 септември) - истинската ширина, не 65°', () => {
  const t = computeCalcDay(day(8, 30), at(69.65, 18.96), 'Turkey', 'shafi');
  const at65 = computeCalcDay(day(8, 30), at(65, 18.96), 'Turkey', 'shafi');
  expect(isValidDay(t)).toBe(true);
  expect(t[4].time.getTime()).not.toBe(at65[4].time.getTime()); // Магриб
});

it('до 65° се смята за истинската ширина (Анкъридж, Икалуит, Ушуая)', () => {
  expect(isPolar(61.22)).toBe(false);
  expect(isPolar(63.75)).toBe(false);
  expect(isPolar(-54.8)).toBe(false);
  for (const [lat, lon] of [
    [61.22, -149.9],
    [63.75, -68.52],
    [-54.8, -68.3],
  ]) {
    expect(isValidDay(computeCalcDay(day(5, 21), at(lat, lon), 'MuslimWorldLeague', 'shafi'))).toBe(true);
    expect(isValidDay(computeCalcDay(day(11, 21), at(lat, lon), 'MuslimWorldLeague', 'shafi'))).toBe(true);
  }
});

it('месец в полярна нощ се смята бързо', () => {
  for (const m of [5, 11]) {

```

```

const t0 = Date.now();
for (let d = 1; d <= 30; d++) computeCalcDay(day(m, d), at(78.22, 15.65), 'Turkey', 'shafi');
console.log(`Свалбард, месец ${m + 1}: ${Date.now() - t0} ms`);
expect(Date.now() - t0).toBeLessThan(300);
}
});
});

```

options.test.ts src/domain/__tests__

- Корекциите местят само своята молитва; в България методът не влияе; извън България влияе; правилото за северни ширини; Тромсьо през юни.

src/domain/__tests__/options.test.ts

60 реда

```

import { DEFAULT_LOCATION } from '@config/defaults';

import { computeCalcDay } from '../calc';
import type { AppLocation } from '../location';
import { computeDay } from '../times';

const date = new Date(2026, 8, 29, 12);
const edinburgh: AppLocation = {
  id: 'edi',
  names: { bg: 'Единбург', en: 'Edinburgh' },
  latitude: 55.9533,
  longitude: -3.1883,
  source: 'calc',
};

describe('ръчни корекции', () => {
  it('+2 мин. на Магриб мести само Магриб', () => {
    const base = computeDay(date, { location: DEFAULT_LOCATION, method: 'Turkey', madhab: 'shafi' });
    const adj = computeDay(date, {
      location: DEFAULT_LOCATION,
      method: 'Turkey',
      madhab: 'shafi',
      offsets: { maghrib: 2 },
    });
    adj.forEach((p, k) => {
      const diff = (p.time.getTime() - base[k].time.getTime()) / 60_000;
      expect(diff).toBe(p.id === 'maghrib' ? 2 : 0);
    });
  });
});

describe('източник и настройки', () => {
  it('в България методът и Асрът не променят часовете (ползва се календарът)', () => {
    const a = computeDay(date, { location: DEFAULT_LOCATION, method: 'Turkey', madhab: 'shafi' });
    const b = computeDay(date, { location: DEFAULT_LOCATION, method: 'MuslimWorldLeague', madhab: 'hanafi' });
    expect(b.map((p) => p.time.getTime())).toEqual(a.map((p) => p.time.getTime()));
  });

  it('извън България методът променя Фаджр (Истанбул: Диянет 18°, Египет 19,5°)', () => {
    // На северни ширини правилото „1/7 от нощта“ често определя Фаджр вместо ъгъла,
    // затова методът се сравнява на нормална ширина.
    const istanbul: AppLocation = { ...edinburgh, id: 'ist', latitude: 41.0082, longitude: 28.9784 };
    const turkey = computeCalcDay(date, istanbul, 'Turkey', 'shafi')[0].time.getTime();
    const egypt = computeCalcDay(date, istanbul, 'Egyptian', 'shafi')[0].time.getTime();
  });
});

```

```

expect(egypt).toBeLessThan(turkey); // по-голям ъгъл → по-ранен Фаджр
});

it('правилото за северни ширини важи над 48°', () => {
  const summer = new Date(2026, 5, 21, 12);
  const seventh = computeCalcDay(summer, edinburgh, 'Turkey', 'shafi', 'seventhofthenight')
[5].time.getTime();
  const middle = computeCalcDay(summer, edinburgh, 'Turkey', 'shafi', 'middleofthenight')
[5].time.getTime();
  expect(seventh).not.toBe(middle);
});

it('над полярния кръг (Тромсьо, юни) пак има часове', () => {
  const tromso: AppLocation = { ...edinburgh, id: 'tos', latitude: 69.6492, longitude: 18.9553 };
  const t = computeCalcDay(new Date(2026, 5, 21, 12), tromso, 'MuslimWorldLeague', 'shafi');
  t.forEach((p) => expect(Number.isNaN(p.time.getTime())).toBe(false));
});
});

```

Трябва да видиш: Tests: 100 passed, 100 total

npm test

Стъпка 9

Проверка и край на етапа

Етап 3 е готов, когато на телефона:

- При първо отваряне се появява въпрос за разрешение за местоположение. След „Разреша“ горе вляво е името на населеното място, в което си.
- „Настройки“ показва мястото с общината и областта и „Календар на Главно мюфтийство“.
- „Настройки“ → „За приложението“ показва версията, източниците и поверителността; „< Настройки“ връща назад.
- В голям град горе пише градът (напр. „София“), а не местност или квартал; в село до града – селото.
- Тап върху името на мястото отваря търсенето. „чеп“ намира двете села Чепинци; изборът сменя часовете на „Днес“ и „Месец“.
- „Използвай местоположението ми (GPS)“ връща автоматичното място.
- Корекция +2 мин. на Магриб се вижда и на „Днес“, и на „Месец“, и остава след рестарт.
- При отказано разрешение: „Няма разрешение за местоположение“ и бутон към настройките на телефона.

Извън България (ако нямаш как да пътуваш): в Android емулятора → ⋮ → Location задай координати, напр. Истанбул 41.0082, 28.9784 или Единбург 55.9533, -3.1883, и натисни „Обнови местоположението“.

Терминал

```

git add .
git commit -m "Етап 3: местоположение, държави и настройки"
git tag stage-3

```

Аср по Ханафи в България: решено да остане само официалният календар. Huawei без Google услуги – в етап 5. Напомнянето преди молитвата и вибрацията са в етап 4, заедно с нотификациите.